

Fase 2: Data Understanding (Entendimento dos Dados)

Nesta fase, é analisado a estrutura e na qualidade das bases transacionais. O objetivo é validar a integridade do faturamento e identificar anomalias sistêmicas antes de qualquer cálculo de ROI para a campanha de lubrificantes.

2.1. Coleta de Dados Brutos

As bases utilizadas foram extraídas do Dashboard Diário de Bordo do PowerBi em formato `.xlsx` e depois convertidas para `.parquet` para otimização de performance e preservação da tipagem. A conversão foi feita através do script `parquet_converter_vendas_devolucoes`:

- `Detalhamento_Vendas.parquet` : Histórico transacional de vendas de peças e serviços.
- `Detalhamento_Devolucoes.parquet` : Registros de estornos e devoluções de mercadoria.
- **Insight:** A unificação dos arquivos anuais permitiu uma visão holística abrangendo o período de setembro/2022 a março/2026., revelando uma base total de 346.622 registros.

In [121...

```
import pandas as pd

# 1. Importação dos arquivos Parquet
df_vendas = pd.read_parquet('Detalhamento_Vendas.parquet')
df_devolucoes = pd.read_parquet('Detalhamento_Devolucoes.parquet')

# 2. Detalhamento Simples do que foi importado
def detalhar_base(df, nome):
    print(f"--- Base: {nome} ---")
    print(f"Dimensões: {df.shape[0]} linhas x {df.shape[1]} colunas")
    print(f"Período: {df.select_dtypes(include=['datetime64']).iloc[:,0].min()}")
    print(f"Colunas Disponíveis: {df.columns.tolist()}\n")

detalhar_base(df_vendas, "Vendas")
detalhar_base(df_devolucoes, "Devoluções")
```

--- Base: Vendas ---

Dimensões: 432951 linhas x 21 colunas

Período: 2022-09-05 00:00:00 até 2026-03-10 00:00:00

Colunas Disponíveis: ['Cód. Produto', 'Descrição', 'Subgrupo', 'Grupo', 'Nota Fiscal', 'Filial', 'Centro Custo', 'Descrição CC', 'Data Emissão', 'Consultor', 'Cód. Cliente', 'CNPJ', 'Cliente', 'Valor Bruto', 'Valor Líquido', 'Impostos', 'Custo', 'Desconto', 'Qtd.', 'ML R\$', 'ML %']

--- Base: Devoluções ---

Dimensões: 8021 linhas x 21 colunas

Período: 2022-09-15 00:00:00 até 2026-03-04 00:00:00

Colunas Disponíveis: ['Cód. Produto', 'Descrição', 'Subgrupo', 'Grupo', 'Nota Fiscal', 'Filial', 'Centro Custo', 'Descrição CC', 'Data', 'Consultor', 'Cód. Cliente', 'CNPJ', 'Cliente', 'Valor Bruto', 'Valor Dev.', 'Imposto', 'Custo', 'Desconto', 'Qtd.', 'ML \$', 'ML %']

2.2. Padronização de Nomenclatura (Snake Case)

Para garantir a escalabilidade do projeto e facilitar as operações de álgebra relacional (Joins e Unions), implementamos um protocolo de nomenclatura padronizado:

- **Snake Case:** Todas as colunas foram convertidas para letras minúsculas e espaços substituídos por underscores (_).
- **Unificação de Chaves:** Colunas que representavam o mesmo conceito mas tinham nomes distintos (ex: `Data Emissão` vs `Data`) foram unificadas sob o mesmo rótulo (`data_emissao`).
- **Semântica de Valor:** Diferenciamos claramente `valor_venda` de `valor_devolucao` para evitar confusões durante o cálculo do Faturamento Líquido (SSS Clean).
- **Remoção de Caracteres Especiais:** Eliminação de pontos, acentos e símbolos monetários nos nomes das variáveis para evitar erros de codificação no Python.

In [122...

```
# 2.2. Dicionário de Tradução/Padronização
# Mapeamos o nome atual para o padrão 'snake_case'
map_vendas = {
    'Cód. Produto': 'cod_produto',
    'Descrição': 'descricao',
    'Subgrupo': 'subgrupo',
    'Grupo': 'grupo',
    'Nota Fiscal': 'nota_fiscal',
    'Filial': 'filial',
    'Centro Custo': 'centro_custo',
    'Descrição CC': 'descricao_cc',
    'Data Emissão': 'data_emissao',
    'Consultor': 'consultor',
    'Cód. Cliente': 'cod_cliente',
    'CNPJ': 'cnpj',
    'Cliente': 'cliente',
    'Valor Bruto': 'valor_bruto',
    'Valor Líquido': 'valor_liquido',
    'Impostos': 'impostos',
    'Custo': 'custo',
    'Desconto': 'desconto',
    'Qtd.': 'quantidade',
```

```

    'ML R$': 'margem_bruta_valor',
    'ML %': 'margem_bruta_perc'
}

map_devolucoes = {
    'Cód. Produto': 'cod_produto',
    'Descrição': 'descricao',
    'Subgrupo': 'subgrupo',
    'Grupo': 'grupo',
    'Nota Fiscal': 'nota_fiscal',
    'Filial': 'filial',
    'Centro Custo': 'centro_custo',
    'Descrição CC': 'descricao_cc',
    'Data': 'data_emissao',           # Unificando com Vendas
    'Consultor': 'consultor',
    'Cód. Cliente': 'cod_cliente',
    'CNPJ': 'cnpj',
    'Cliente': 'cliente',
    'Valor Bruto': 'valor_bruto_dev',
    'Valor Dev.': 'valor_liquido',
    'Imposto': 'impostos',           # Unificando plural
    'Custo': 'custo',
    'Desconto': 'desconto',
    'Qtd.': 'quantidade',
    'ML $': 'margem_bruta_valor',    # Unificando símbolo
    'ML %': 'margem_bruta_perc'
}

# 2. Aplicação do Rename
df_vendas.rename(columns=map_vendas, inplace=True)
df_devolucoes.rename(columns=map_devolucoes, inplace=True)

print("✅ Colunas padronizadas.")
print(f"Colunas Vendas: {df_vendas.columns.tolist()[:5]}...")
print(f"Colunas Devoluções: {df_devolucoes.columns.tolist()[:5]}...")

```

✅ Colunas padronizadas.

Colunas Vendas: ['cod_produto', 'descricao', 'subgrupo', 'grupo', 'nota_fiscal']...

Colunas Devoluções: ['cod_produto', 'descricao', 'subgrupo', 'grupo', 'nota_fiscal']...

2.3. Análise de Tipagem e Formatos (Data Schema)

Realizamos a auditoria dos tipos primitivos de dados para assegurar que identificadores numéricos não fossem tratados como variáveis contínuas. As colunas de identificação estavam sendo tratadas como `float64`, o que gera riscos de perda de precisão e formatação incorreta (ex: `8.96e+13`).

- **Colunas Alteradas:** `nota_fiscal`, `centro_custo`, `cod_cliente` e `cnpj`.
- **Ação Técnica:** Conversão de numérico para `string` (object), com remoção de sufixos decimais (`.0`) e tratamento de valores nulos para evitar strings `'nan'`.
- **Objetivo:** Isolar estas colunas da análise estatística (média/mediana) e garantir a integridade da chave primária para o cruzamento com a base de Potencial.

In [123...

```
## 2.3. Análise de Tipagem e Formatos (Data Schema)

# Verificando detalhadamente os tipos de dados e nomes de colunas
print("--- Tipagem: Vendas ---")
print(df_vendas.dtypes)

print("\n--- Tipagem: Devoluções ---")
print(df_devolucoes.dtypes)

# Verificação específica da chave de Join (CNPJ)
# Se estiver como float, o Python pode converter 00123... para 123.0 (perda de z
sample_cnpj = df_vendas['cnpj'].dropna().iloc[0]
print(f"\nExemplo de formato cnpj atual: {sample_cnpj} (Tipo: {type(sample_cnpj)})")
```

```

--- Tipagem: Vendas ---
cod_produto          object
descricao            object
subgrupo            object
grupo              object
nota_fiscal         float64
filial              object
centro_custo        float64
descricao_cc        object
data_emissao        datetime64[ns]
consultor           object
cod_cliente         float64
cnpj                float64
cliente             object
valor_bruto         float64
valor_liquido       float64
impostos            float64
custo               float64
desconto            float64
quantidade         float64
margem_bruta_valor  float64
margem_bruta_perc   float64
dtype: object

```

```

--- Tipagem: Devoluções ---
cod_produto          object
descricao            object
subgrupo            object
grupo              object
nota_fiscal         float64
filial              object
centro_custo        float64
descricao_cc        object
data_emissao        datetime64[ns]
consultor           object
cod_cliente         float64
cnpj                float64
cliente             object
valor_bruto_dev     float64
valor_liquido       float64
impostos            float64
custo               float64
desconto            float64
quantidade         float64
margem_bruta_valor  float64
margem_bruta_perc   float64
dtype: object

```

Exemplo de formato cnpj atual: 89674782001553.0 (Tipo: <class 'numpy.float64'>)

2.3.1. Refinamento de Tipagem (Casting de IDs)

In [124...

```

## 2.3.1. Refinamento de Tipagem (Casting de IDs)

import pandas as pd

# 1. Definição das colunas que devem ser tratadas como ID (Objeto)
cols_to_object = ['nota_fiscal', 'centro_custo', 'cod_cliente', 'cnpj']

```

```
# 2. Aplicação da conversão em ambos os datasets
for df in [df_vendas, df_devolucoes]:
    for col in cols_to_object:
        if col in df.columns:
            # Primeiro removemos o .0 (caso exista por ser float) e convertemos
            df[col] = df[col].astype(str).str.replace('\.0$', '', regex=True)
            # Substituímos 'nan' textual (gerado pelo astype em nulos) por None
            df[col] = df[col].replace('nan', None)

print("✅ Tipagem atualizada com sucesso.")

# 3. Verificação rápida para confirmar se saíram do grupo numérico
print("\n--- Novos tipos em Vendas ---")
print(df_vendas[cols_to_object].dtypes)
```

✅ Tipagem atualizada com sucesso.

```
--- Novos tipos em Vendas ---
nota_fiscal    object
centro_custo    object
cod_cliente    object
cnpj           object
dtype: object
```

2.4. Qualidade de Dados: Diagnóstico de Nulos

Realizamos uma varredura em todas as colunas para identificar o "vazio" de informações. Este diagnóstico é vital para a governança dos dados da INOVA:

Resultado da Auditoria

- **Ponto Crítico:** A coluna `subgrupo` (driver principal para identificar Lubrificantes) apresenta **61,70% de nulos**. Isso sugere que a análise por aplicação (ex: Peças de Motor vs. Transmissão) dependerá da descrição do produto como base para recuperação de categorias
- **Estabilidade:** Variáveis financeiras como `valor_liquido` e `custo` apresentam preenchimento próximo de 100%, garantindo a confiabilidade das somatórias globais.
- **Chaves de Identificação:** As colunas `cnpj` e `cod_cliente` apresentam baixíssimo índice de nulos. Estes serão tratados na Fase 3, pois são a âncora do nosso projeto.
- **Variáveis de Valor:** As métricas financeiras (`valor_venda`, `custo`, `margem`) estão praticamente 100% preenchidas, garantindo a confiabilidade do cálculo do faturamento líquido.

In [125...

```
## 2.4. Qualidade de Dados: Diagnóstico de Nulos

import pandas as pd

def report_qualidade_nulos(df, nome):
    # Cálculo de preenchimento
    total_linhas = len(df)
    nulos = df.isnull().sum()
    percentual_nulos = (nulos / total_linhas) * 100
```

```

# Criando o DataFrame de relatório
df_quality = pd.DataFrame({
    'coluna': df.columns,
    'registros_preenchidos': df.count().values,
    'qtd_nulos': nulos.values,
    '%_nulos': percentual_nulos.values
}).sort_values('%_nulos', ascending=False)

print(f"--- Relatório de Qualidade: {nome} ---")
# Estilização: Cores mais intensas onde há mais nulos
return df_quality.style.background_gradient(cmap='YlOrRd', subset=['%_nulos'])

# Execução para as bases padronizadas
display(report_qualidade_nulos(df_vendas, "Vendas"))
display(report_qualidade_nulos(df_devolucoes, "Devoluções"))

```

--- Relatório de Qualidade: Vendas ---

	coluna	registros_preenchidos	qtd_nulos	%_nulos
2	subgrupo	162343	270608	62.50%
6	centro_custo	234658	198293	45.80%
7	descricao_cc	234658	198293	45.80%
1	descricao	429320	3631	0.84%
3	grupo	429320	3631	0.84%
0	cod_produto	429328	3623	0.84%
5	filial	432941	10	0.00%
12	cliente	432942	9	0.00%
11	cnpi	432942	9	0.00%
10	cod_cliente	432942	9	0.00%
9	consultor	432942	9	0.00%
8	data_emissao	432942	9	0.00%
4	nota_fiscal	432942	9	0.00%
13	valor_bruto	432946	5	0.00%
14	valor_liquido	432946	5	0.00%
15	impostos	432946	5	0.00%
16	custo	432946	5	0.00%
17	desconto	432946	5	0.00%
18	quantidade	432946	5	0.00%
19	margem_bruta_valor	432946	5	0.00%
20	margem_bruta_perc	432946	5	0.00%

--- Relatório de Qualidade: Devoluções ---

	coluna	registros_preenchidos	qtd_nulos	%_nulos
2	subgrupo	2945	5076	63.28%
6	centro_custo	5570	2451	30.56%
7	descricao_cc	5570	2451	30.56%
3	grupo	8002	19	0.24%
1	descricao	8002	19	0.24%
0	cod_produto	8004	17	0.21%
9	consultor	8018	3	0.04%
12	cliente	8018	3	0.04%
11	cnpj	8018	3	0.04%
10	cod_cliente	8018	3	0.04%
8	data_emissao	8018	3	0.04%
5	filial	8018	3	0.04%
4	nota_fiscal	8018	3	0.04%
13	valor_bruto_dev	8019	2	0.02%
14	valor_liquido	8019	2	0.02%
15	impostos	8019	2	0.02%
16	custo	8019	2	0.02%
17	desconto	8019	2	0.02%
18	quantidade	8019	2	0.02%
19	margem_bruta_valor	8019	2	0.02%
20	margem_bruta_perc	8019	2	0.02%

2.5. Qualidade de Dados: Auditoria de Duplicidade

Aplicamos um rigoroso teste de unicidade transacional para identificar registros redundantes que pudessem inflar o faturamento real da companhia.

- **Premissa de Unicidade:** Cada registro deve ser único para a combinação de **Nota Fiscal + Código do Produto**.
- **Resultado da Auditoria:** Nenhuma duplicidade exata encontrada.

In [126...

```
## 2.5. Qualidade de Dados: Auditoria de Duplicidade
```

```
import pandas as pd
```

```
def report_duplicados_sequencial(df, nome):
    print(f"--- 🐞 Auditoria de Duplicados: {nome} ---")
```



```

# 1. Cálculo de Métricas Básicas
total_linhas = len(df)
# Identificação de TODAS as cópias (keep=False)
df_dups = df[df.duplicated(keep=False)].copy()
qtd_dups = df.duplicated().sum() # Apenas o "excesso"

if not df_dups.empty:
    # 2. Ordenação Estratégica para Agrupamento Visual
    # Usamos nota_fiscal e cod_produto como âncoras de ordenação
    # Note: 'valor_liquido' já está normalizado em ambas conforme sua ação
    cols_sort = ['nota_fiscal', 'cod_produto', 'valor_liquido', 'quantidade']
    cols_sort = [c for c in cols_sort if c in df_dups.columns]

    df_dups_sorted = df_dups.sort_values(by=cols_sort)

    print(f"Total de Linhas na Base: {total_linhas}")
    print(f"Registros redundantes (Excesso): {qtd_dups} ({(qtd_dups/total_linhas)*100:.1f}%)")
    print(f"Total de linhas envolvidas em duplicidade: {len(df_dups)}")
    print(f"\n🔍 Amostra de 'Gêmeos Idênticos' em {nome} (sequencial):")
    display(df_dups_sorted.head(10))
else:
    print(f"✅ {nome}: Nenhuma duplicidade exata encontrada.")
    print("\n" + "="*60 + "\n")

# Execução para ambos os datasets
report_duplicados_sequencial(df_vendas, "Vendas")
report_duplicados_sequencial(df_devolucoes, "Devoluções")

```

```

--- 🧑‍🔬 Auditoria de Duplicados: Vendas ---

```

```

✅ Vendas: Nenhuma duplicidade exata encontrada.

```

```

=====

```

```

--- 🧑‍🔬 Auditoria de Duplicados: Devoluções ---

```

```

✅ Devoluções: Nenhuma duplicidade exata encontrada.

```

```

=====

```

2.6. Auditoria de Estrutura (Data Description Report)

Separação das colunas em três grupos fundamentais. Essa taxonomia permite definir as técnicas de EDA que usaremos a seguir:

- Colunas Numéricas (Quantitativas):** Utilizadas para cálculos de faturamento líquido, margem de contribuição (ML) e descontos.
- Colunas de Data (Temporais):** Essenciais para identificar a "Recência" do cliente (RFM) e janelas de Churn na Linha Amarela.
- Colunas de Objeto (Qualitativas/Categoriais):** Contêm metadados como CNPJ, Descrição de Peças e Segmentação de Consultores.

Resultado da Auditoria

- **Consistência Identificada:** Ambas as bases possuem a mesma largura de colunas (21), facilitando operações de `concat` se necessário no futuro.
- **Validação de Chave:** Confirmamos que `cnnpj` e `cod_cliente` estão no grupo numérico (`float64`), o que reforça a necessidade de tratamento de tipos na Fase 3 para evitar perda de integridade.

In [127...

```
## 2.6. Auditoria de Estrutura (Data Description Report)

import pandas as pd

# 1. Lista de datasets e seus títulos
datasets = [df_vendas, df_devolucoes]
titles = ["Vendas", "Devoluções"]

# 2. Definição dos tipos para auditoria
numerics = ['int16', 'int32', 'int64', 'float16', 'float32', 'float64']
dates = ['datetime64', 'datetime64[ns]']

# 3. Construção do DataFrame de Auditoria de Tipos
df_types_check = pd.DataFrame({'dataset': titles})

# Colunas Numéricas
df_types_check['numeric_count'] = [len(df.select_dtypes(include=numerics).columns) for df in datasets]
df_types_check['numeric_cols'] = [', '.join(df.select_dtypes(include=numerics).columns) for df in datasets]

# Colunas de Data
df_types_check['date_count'] = [len(df.select_dtypes(include=dates).columns) for df in datasets]
df_types_check['date_cols'] = [', '.join(df.select_dtypes(include=dates).columns) for df in datasets]

# Colunas de Objeto (Texto/Categorias)
df_types_check['object_count'] = [len(df.select_dtypes(include='object').columns) for df in datasets]
df_types_check['object_cols'] = [', '.join(df.select_dtypes(include='object').columns) for df in datasets]

# 4. Exibição com Estilização para facilitar a leitura técnica
display(df_types_check.style.background_gradient(cmap='Greens', subset=['numeric_count', 'numeric_cols', 'date_count', 'date_cols', 'object_count', 'object_cols']))
```

	dataset	numeric_count	numeric_cols	date_count	date_cols	object_count
0	Vendas	8	valor_bruto, valor_liquido, impostos, custo, desconto, quantidade, margem_bruta_valor, margem_bruta_perc	1	data_emissao	12
1	Devoluções	8	valor_bruto_dev, valor_liquido, impostos, custo, desconto, quantidade, margem_bruta_valor, margem_bruta_perc	1	data_emissao	12

2.6.1.1. Diagnóstico de Colunas Numéricas

A exploração estatística revelou outliers extremos que indicam erros de integração ou unidades de medida incorretas no ERP:

- 1. **Outliers de Faturamento:** O `valor_liquido` máximo é de **R\$ 419 Milhões** em uma única linha, enquanto 75% da base gasta até **R\$ 769,12**. Isso não é uma venda comum, é provavelmente um erro de sistema ou um lançamento de ajuste contábil.
- 2. **Erros de Quantidade:** A `quantidade` máxima de **1.285.539** unidades em um único registro sugere erro de unidade de medida (ex: converter litros em gotas ou gramas em miligramas indevidamente).
- 3. **Margens Impossíveis:** * Temos `margem_bruta_perc` mínima de **-16.296%**. Isso indica que o custo registrado é absurdamente maior que a venda, ou um erro de digitação.
 - A `margem_bruta_valor` mínima de **-R\$ 234 mil** em uma linha de venda sugere prejuízo operacional grave ou erro de custo.
- 4. **Assimetria (Skewness):** A média de faturamento (3.480) é quase **12 vezes maior** que a mediana (282), confirmando que outliers extremos estão "puxando" os dados.

In [128...

```
## 2.6.1.1 Diagnóstico de Colunas Numéricas

import pandas as pd

# 1. Seleção de métricas de interesse (usando os nomes padronizados)
metrics_vendas = ['valor_liquido', 'quantidade', 'margem_bruta_valor', 'margem_b
```

```
metrics_dev = ['valor_devolucao', 'quantidade'] # Ajustado para o nome da base d

def gerar_tabela_estadistica(df, title, columns):
    print(f"--- 📊 Tabela de Auditoria Estatística: {title} ---")

    # Filtra apenas as colunas que existem no DF para evitar erro
    cols_existentes = [c for c in columns if c in df.columns]

    # Geração das estatísticas descritivas
    stats = df[cols_existentes].describe().T

    # Cálculo do Coeficiente de Variação (CV)
    # CV > 1 indica alta dispersão
    stats['cv'] = stats['std'] / stats['mean']

    # Exibição estilizada com separador de milhar e 2 casas decimais
    display(stats.style.format("{:,.2f}").background_gradient(cmap='Greens', sub

# Execução
gerar_tabela_estadistica(df_vendas, "Vendas", metrics_vendas)
gerar_tabela_estadistica(df_devolucoes, "Devoluções", metrics_dev)
```

--- 📊 Tabela de Auditoria Estatística: Vendas ---

	count	mean	std	min	25%	50%	75%
valor_liquido	432,946.00	3,017.94	546,712.92	0.01	102.78	289.39	802.59
quantidade	432,946.00	8.66	1,562.36	0.70	1.00	1.00	2.00
margem_bruta_valor	432,946.00	889.89	161,709.34	-234,868.06	23.53	90.72	256.77
margem_bruta_perc	432,946.00	0.28	25.04	-16,296.00	0.26	0.35	0.42



--- 📊 Tabela de Auditoria Estatística: Devoluções ---

	count	mean	std	min	25%	50%	75%	max	cv
quantidade	8,019.00	8.40	376.83	1.00	1.00	1.00	2.00	33,687.67	44.85

2.6.1.2. Descoberta de Erros Estruturais (Linhas de Totalização)

- **Outliers Extremos** Durante a inspeção profunda, identificamos que os outliers extremos (ex: R\$ 419M) não são transações reais, mas sim **linhas de rodapé ("Total")** inseridas pelo sistema de extração.
- **Impacto:** Essas linhas duplicam o faturamento e as quantidades, tornando qualquer média ou soma simples inválida.
- **Higiene de Vendas:** Identificamos registros da própria 'INOVA MAQUINAS LTDA' como cliente, o que sugere transferências internas ou lançamentos de ajuste que não devem compor a análise de faturamento de clientes externos.
- **Validação de Margens:** Confirmamos que margens negativas são aceitáveis como fenômeno de negócio (prejuízo na venda), mas valores líquidos negativos na base de vendas serão tratados como inconsistência sistêmica.

- **Métrica de Dispersão (CV):** O Coeficiente de Variação é utilizado como KPI de qualidade da base. Valores de CV extremamente elevados (ex: > 100) ratificam a presença de outliers que não pertencem à distribuição normal de vendas de peças.

In [129...

```
## 2.6.1.2. Descoberta de Erros Estruturais (Linhas de Totalização)

import pandas as pd

# 1. Identificando as 10 maiores transações com todas as colunas
outliers_faturamento = df_vendas.nlargest(10, 'valor_liquido')

print("--- 🚨 Top 10 Transações Suspeitas (Visão Completa) ---")
# display(outliers_faturamento) exibirá todas as colunas no Jupyter
display(outliers_faturamento)

# 2. Identificando Margens Irreais com Visão Completa
# Filtro: Margens menores que -100% ou maiores que 100%
margens_suspeitas = df_vendas[(df_vendas['margem_bruta_perc'] < -1) | (df_vendas

print(f"\n🚨 Total de registros com margens irreais: {len(margens_suspeitas)}")

if len(margens_suspeitas) > 0:
    print("--- Amostra de Registros com Margens Inconsistentes ---")
    # Exibindo os 10 casos mais críticos de margem negativa
    display(margens_suspeitas.sort_values('margem_bruta_perc').head(10))
```

--- 🚨 Top 10 Transações Suspeitas (Visão Completa) ---

	cod_produto	descricao	subgrupo	grupo	nota_fiscal	filial	centro_cust
	146119	Total	None	None	None	None	Nor
	409999	Total	None	None	None	None	Nor
	271886	Total	None	None	None	None	Nor
	432949	Total	None	None	None	None	Nor
	13506	None	None	None	9	0201 - Contagem	Nor
	13542	None	None	None	111419	0201 - Contagem	Nor
	13524	None	None	None	30	0201 - Contagem	Nor
	209023	9299930 CONJ. PLANETARIOS	None	JDPC	154812	0212 - CSN	101031
	13529	None	None	None	35	0201 - Contagem	Nor
	13530	None	None	None	36	0201 - Contagem	Nor

10 rows × 21 columns



⚠ Total de registros com margens irreais: 769
--- Amostra de Registros com Margens Inconsistentes ---

	cod_produto	descricao	subgrupo	grupo	nota_fiscal	filial	centr
155562	22AC16224	COPO TRMICO JOHN DEERE 400 ML - PRETO	None	JDCO	159528	0201 - Contagem	
5705	CQM20196	AIR CLEAN JOHN DEERE HIGIENIZA	None	CLGD	123201	0201 - Contagem	
19386	CQM20196	AIR CLEAN JOHN DEERE HIGIENIZA	None	CLGD	122196	0201 - Contagem	
5704	CQM20196	AIR CLEAN JOHN DEERE HIGIENIZA	None	CLGD	122823	0201 - Contagem	
136968	CQM20196	AIR CLEAN JOHN DEERE HIGIENIZA	None	CLGD	2226	0204 - Urberlândia	
347202	CQM20131	OLEO HY- GARD 1000L TRANSMISSAO	LUBRIFICANTE	LUB	16635	0204 - Urberlândia	
153023	AT340027P	MANG MONTADA COLETA TANQUE HIDRAULICO - MANG M...	None	JDOU	196	0212 - CSN	
17843	AT318789	KIT RETENTOR	None	JDPC	133895	0201 - Contagem	
398453	T187987	FACA CORTE DURA-MAX	FPS	JDPC	212935	0212 - CSN	
398452	T187987	FACA CORTE DURA-MAX	FPS	JDPC	212933	0212 - CSN	

10 rows × 21 columns



2.6.2.1 Diagnóstico de Volumetria Histórica

A análise temporal revelou três janelas distintas de comportamento na base de dados da INOVA:

- 1. **Período de Maturação (2022-2023):** Crescimento gradual de transações, saindo de 7k para 11k registros/mês.

2. **Pico Operacional (2024):** Estabilidade em patamares elevados (~18k registros/mês). Este é o nosso "Baseline" de alta performance para a Linha Amarela.
3. **Anomalia de Volume (2025-2026):** Queda abrupta para ~5k registros.
 - **Risco Identificado:** Se esta queda for sistêmica (falha na extração), as métricas de Churn estarão superestimadas. Se for real, indica uma retração severa de mercado ou perda de market share para peças paralelas.
 - **Devoluções:** Enquanto as vendas caíram ~60%, as devoluções mantiveram-se estáveis ou cresceram em meses específicos (Ex: 2025-04 com 378 registros), indicando uma piora na qualidade da venda ou erro logístico.

Implicação para o TCO e Potencial

Para o cálculo do potencial, utilizaremos o ano de 2024 como referência de comportamento de consumo ideal, tratando 2025 como um período de "atenção seletiva" até validarmos se os dados estão completos.

In [130...

```
## 2.6.2.1 Diagnóstico de Volumetria Histórica

import pandas as pd

def eda_temporal(df, title):
    print(f"--- 📊 Auditoria Temporal: {title} ---")

    # 1. Amplitude do Histórico
    data_min = df['data_emissao'].min()
    data_max = df['data_emissao'].max()
    amplitude = data_max - data_min

    # 2. Agrupamento Mensal para ver volumetria
    # Criamos uma coluna temporária de Ano-Mês para a contagem
    df['ano_mes'] = df['data_emissao'].dt.to_period('M')
    vendas_mensais = df.groupby('ano_mes').size()

    print(f"Início do Histórico: {data_min}")
    print(f"Fim do Histórico: {data_max}")
    print(f"Janela de Tempo: {amplitude.days} dias")
    print(f"\n📦 Volume de Transações por Mês:")
    print(vendas_mensais)

    # Removendo a coluna temporária para manter o DF Limpo
    df.drop(columns=['ano_mes'], inplace=True)
    print("-" * 50)

# Execução
eda_temporal(df_vendas, "Vendas")
eda_temporal(df_devolucoes, "Devoluções")
```


--- 📅 Auditoria Temporal: Vendas ---
Início do Histórico: 2022-09-05 00:00:00
Fim do Histórico: 2026-03-10 00:00:00
Janela de Tempo: 1282 dias

📦 Volume de Transações por Mês:

ano_mes	
2022-09	7038
2022-10	9216
2022-11	7535
2022-12	7624
2023-01	8063
2023-02	7363
2023-03	10738
2023-04	8228
2023-05	11398
2023-06	10161
2023-07	10073
2023-08	11115
2023-09	10752
2023-10	8969
2023-11	9180
2023-12	8666
2024-01	9216
2024-02	8706
2024-03	9536
2024-04	11480
2024-05	11484
2024-06	10620
2024-07	10960
2024-08	11285
2024-09	11854
2024-10	11055
2024-11	10242
2024-12	9326
2025-01	10304
2025-02	11002
2025-03	10397
2025-04	11894
2025-05	12177
2025-06	11427
2025-07	12847
2025-08	12479
2025-09	13091
2025-10	12325
2025-11	9713
2025-12	10455
2026-01	10689
2026-02	9779
2026-03	2480

Freq: M, dtype: int64

--- 📅 Auditoria Temporal: Devoluções ---
Início do Histórico: 2022-09-15 00:00:00
Fim do Histórico: 2026-03-04 00:00:00
Janela de Tempo: 1266 dias

📦 Volume de Transações por Mês:

ano_mes	
2022-09	9

```

2022-10    107
2022-11     96
2022-12    145
2023-01    140
2023-02    219
2023-03    120
2023-04    187
2023-05    171
2023-06    106
2023-07    221
2023-08    218
2023-09    253
2023-10    172
2023-11     93
2023-12    229
2024-01    200
2024-02    145
2024-03    114
2024-04    165
2024-05    138
2024-06    237
2024-07    219
2024-08    155
2024-09    127
2024-10    194
2024-11    348
2024-12    250
2025-01    162
2025-02    256
2025-03    158
2025-04    378
2025-05    164
2025-06    234
2025-07    156
2025-08    136
2025-09    268
2025-10    277
2025-11    281
2025-12    194
2026-01    217
2026-02    338
2026-03     21
Freq: M, dtype: int64
-----

```

2.6.2.2 Diagnóstico de Volumetria Histórica

Análise temporal dos registros para certificar que não houve gap entre as extrações do PowerBI. As únicas datas sem vendas estão nos finais de semanas e feriados.

```

In [131...  ## 2.6.2.2 Diagnóstico de Volumetria Histórica

def check_date_gaps(df, title):
    # Criar um range completo de datas do início ao fim
    full_range = pd.date_range(start=df['data_emissao'].min(), end=df['data_emis
    # Datas que existem no dataframe
    dates_in_df = df['data_emissao'].dt.normalize().unique()
    # Identificar datas faltantes

```

```

missing_dates = full_range.difference(dates_in_df)

print(f"--- Gaps de Datas: {title} ---")
print(f"Total de dias no período: {len(full_range)}")
print(f"Dias sem NENHUM registro: {len(missing_dates)}")
if len(missing_dates) > 0:
    print(f"Exemplos de dias sem dados: {missing_dates[:5].tolist()}")
print("-" * 50)

check_date_gaps(df_vendas, "Vendas")

```

```

--- Gaps de Datas: Vendas ---
Total de dias no período: 1283
Dias sem NENHUM registro: 312
Exemplos de dias sem dados: [Timestamp('2022-09-07 00:00:00'), Timestamp('2022-09-11 00:00:00'), Timestamp('2022-09-17 00:00:00'), Timestamp('2022-09-18 00:00:00'), Timestamp('2022-09-24 00:00:00')]
-----

```

2.6.3. Análise de Variáveis Categóricas (Cardinalidade e Perfil)

- **Cardinalidade de Produtos:** O número de `cod_produto` únicos revela a profundidade do estoque movimentado. Uma concentração muito alta em poucos códigos (ex: Óleo Plus-50 II) valida a estratégia de focar em Lubrificantes..
- **Presença Geográfica (Filiais):** A análise identifica as unidades com maior volume transacional, permitindo que a INOVA direcione esforços logísticos e consultores de campo para as regiões de maior "calor" de vendas.
- **Estrutura de Atendimento (Consultores):** Mapeamos a distribuição da carga de trabalho. Consultores com excesso de transações podem ter dificuldade em realizar o "contato ativo" necessário para converter o potencial de TCO.
- **Perfil da Carteira (Clientes):** A identificação do `% de Concentração` do maior cliente ajuda a mitigar riscos. Se um único CNPJ representa uma fatia muito grande das transações, a perda desse cliente (Churn) teria um impacto catastrófico na margem da filial.

In [132...

```

## 2.6.3. Análise de Variáveis Categóricas (Cardinalidade e Perfil)

import pandas as pd

# 1. Definição das colunas de objeto para análise (Incluindo 'cnpj' para auditoria)
cols_objeto = ['filial', 'grupo', 'subgrupo', 'consultor', 'cliente', 'cnpj', 'c']

def eda_categorica(df, title):
    print(f"--- 📁 Auditoria de Categorias: {title} ---")

    report_data = []
    total_linhas = len(df)

    for col in cols_objeto:
        if col in df.columns:
            # Calculando métricas de frequência e diversidade
            n_unique = df[col].nunique()
            counts = df[col].value_counts()

```

```

if not counts.empty:
    top_val = counts.idxmax()
    top_freq = counts.max()
    perc_top = (top_freq / total_linhas) * 100

    report_data.append({
        'Coluna': col,
        'Valores Únicos': n_unique,
        'Valor Mais Frequente': top_val,
        'Frequência': top_freq,
        '% de Concentração': f"{perc_top:.2f}%"
    })

df_report = pd.DataFrame(report_data)

# Estilização para destacar alta cardinalidade (azul)
# Importante para identificar colunas que podem ser chaves primárias ou nome
display(df_report.style.background_gradient(cmap='Blues', subset=['Valores Ú
    .set_caption(f"Análise de Cardinalidade - {title}"))

# Insight Proativo de Arquiteto para a Etapa 2.6.3
if 'cliente' in df.columns and 'cnpj' in df.columns:
    dif = df['cliente'].nunique() - df['cnpj'].nunique()
    if dif > 0:
        print(f"💡 Insight EDA: Existem {dif} variações de nomes a mais que
        print(f"Isso indica prováveis duplicidades de cadastro que serão san

print("-" * 60 + "\n")

# Execução nos datasets originais (Fase 2 - EDA)
eda_categorica(df_vendas, "Vendas")
eda_categorica(df_devolucoes, "Devoluções")

```

--- 🛒 Auditoria de Categorias: Vendas ---

Análise de Cardinalidade - Vendas

	Coluna	Valores Únicos	Valor Mais Frequente	Frequência	% de Concentração
0	filial	9	0201 - Contagem	188805	43.61%
1	grupo	31	JDPC	382270	88.29%
2	subgrupo	5	FILTROS	133553	30.85%
3	consultor	101	Lilian Queiroz	81365	18.79%
4	cliente	3971	CSN MINERACAO S.A.	106307	24.55%
5	cnpj	3988	8902291000468	69507	16.05%
6	cod_produto	20568	RE541922	10219	2.36%

--- 🛒 Auditoria de Categorias: Devoluções ---

Análise de Cardinalidade - Devoluções

	Coluna	Valores Únicos	Valor Mais Frequente	Frequência	% de Concentração
0	filial	7	0201 - Contagem	3282	40.92%
1	grupo	21	JDPC	7051	87.91%
2	subgrupo	5	FILTROS	2304	28.72%
3	consultor	70	Lilian Queiroz	922	11.49%
4	cliente	762	CSN MINERACAO S.A.	1412	17.60%
5	cnpj	762	8902291000468	1412	17.60%
6	cod_produto	3223	CQM20204	169	2.11%

In [133...

```
## 2.6.4. Auditoria de Integridade da Chave Primária (CNPJ)

def auditar_cnpj(df, nome_base):
    print(f"--- 🚫 Auditoria de Qualidade CNPJ: {nome_base} ---")

    # Garantindo que estamos trabalhando com strings e tratando valores nulos
    temp_cnpj = df['cnpj'].astype(str).replace('None', 'Nulo')
    total_registros = len(temp_cnpj)

    # 1. Verificar caracteres não numéricos
    # Identifica o que não é dígito (excluindo o marcador 'Nulo' que criamos aci
    caracteres_estranhos = temp_cnpj[~temp_cnpj.str.contains(r'^\d+$', na=False)]

    # 2. Verificar comprimentos e calcular porcentagens
    contagem_absoluta = temp_cnpj.str.len().value_counts()
    contagem_percentual = (temp_cnpj.str.len().value_counts(normalize=True) * 100).sort_index(ascending=False)

    # Criando um DataFrame para exibição tabular organizada
    df_distribuicao = pd.DataFrame({
        'Quantidade': contagem_absoluta,
        'Porcentagem (%)': contagem_percentual.map('{:.2f}%'.format)
    }).sort_index(ascending=False)

    # 3. Identificar padrões suspeitos para enriquecimento futuro
    # CNPJs/CPFs válidos costumam ter 11 ou 14 dígitos (considerando que o zero
    # Registros com < 10 dígitos são fortíssimos candidatos a erro de cadastro
    cnpjs_criticos = temp_cnpj[temp_cnpj.str.len() < 10].unique()

    print(f"✅ Caracteres não-numéricos detectados: {caracteres_estranhos}")
    print(f"✅ Distribuição de comprimentos (Qualidade da Coluna):")
    print(df_distribuicao)

    if len(cnpjs_criticos) > 0:
        print(f"\n⚠️ Alerta: {len(cnpjs_criticos)} padrões com tamanho crítico")
        print(f"💡 Exemplos para correção na Fase 3: {cnpjs_criticos[:5]}")
    else:
        print("\n✅ Nenhum padrão de comprimento crítico detectado.")
    print("-" * 60 + "\n")

# Executando a auditoria nas bases
```

```
auditar_cnpj(df_vendas, "Vendas")
auditar_cnpj(df_devolucoes, "Devoluções")
```

--- 🟡 Auditoria de Qualidade CNPJ: Vendas ---

- ✓ Caracteres não-numéricos detectados: []
- ✓ Distribuição de comprimentos (Qualidade da Coluna):

	Quantidade	Porcentagem (%)
cnpj		
14	204276	47.18%
13	179215	41.39%
12	7649	1.77%
11	23084	5.33%
10	16556	3.82%
9	2125	0.49%
8	36	0.01%
7	1	0.00%
4	9	0.00%

⚠️ Alerta: 58 padrões com tamanho crítico (< 10 dígitos) identificados.

💡 Exemplos para correção na Fase 3: ['542515830' '826782604' '918943604' '256403643' '508381649']

--- 🟡 Auditoria de Qualidade CNPJ: Devoluções ---

- ✓ Caracteres não-numéricos detectados: []
- ✓ Distribuição de comprimentos (Qualidade da Coluna):

	Quantidade	Porcentagem (%)
cnpj		
14	4059	50.60%
13	3113	38.81%
12	215	2.68%
11	369	4.60%
10	256	3.19%
9	6	0.07%
4	3	0.04%

⚠️ Alerta: 6 padrões com tamanho crítico (< 10 dígitos) identificados.

💡 Exemplos para correção na Fase 3: ['401904202' '182280624' '169904717' '152906703' '303507608']

2.7. Comparativo de Tendência (Vendas vs. Devoluções)

Para comparar grandezas em escalas distintas (Faturamento vs. Volume de Estornos), aplicamos a normalização **Min-Max**. Este método isola o valor absoluto e foca no **comportamento da curva**:

- **Higiene Visual:** Foram removidos os registros de "Total" (outliers sistêmicos) para permitir a visualização da oscilação real do dia a dia das filiais.
- **Análise de 2024:** Este ano serve como nosso *baseline* de alta performance. O gráfico permite visualizar se o aumento de faturamento gerou uma pressão logística que resultou em mais devoluções.
- **Análise de 2025/2026:** A normalização ajuda a identificar se a queda de faturamento observada no diagnóstico anterior é acompanhada por uma queda

proporcional nas devoluções ou se estas permanecem altas (o que indicaria uma deterioração da margem líquida).

Insight Esperado:

Se a curva de devoluções (vermelha) estiver frequentemente acima da de vendas (verde) em termos de variação, temos um sinal de alerta no processo de expedição ou na qualidade das peças (Peças Genuínas vs. Erros de Catálogo).

In [134...

```
## 2.7. Comparativo de Tendência (Vendas vs. Devoluções)

import matplotlib.pyplot as plt
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# 1. Preparação dos Dados
# Filtramos 'Total' e garantimos que trabalhamos com cópias para evitar SettingWithCopyWarning
df_v_plot = df_vendas[df_vendas['cod_produto'] != 'Total'].copy()
df_d_plot = df_devolucoes[df_devolucoes['cod_produto'] != 'Total'].copy()

# 2. Agrupamento mensal usando 'ME' (Month End) conforme a nova documentação do Pandas
# Nota: corrigido 'valor_liquido' para 'valor_liquido' na base de estornos
vendas_mes = df_v_plot.resample('ME', on='data_emissao')['valor_liquido'].sum()
dev_mes = df_d_plot.resample('ME', on='data_emissao')['valor_liquido'].sum()

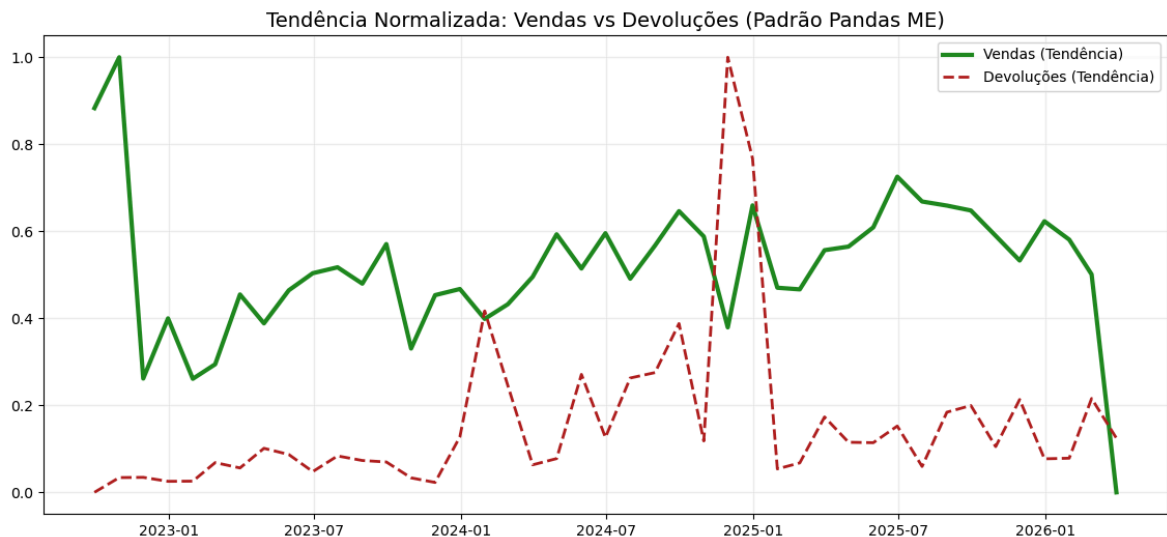
# 3. Alinhamento e Normalização
df_timeline = pd.DataFrame({'vendas': vendas_mes, 'devolucoes': dev_mes}).fillna(0)

scaler = MinMaxScaler()
df_timeline_norm = pd.DataFrame(
    scaler.fit_transform(df_timeline),
    index=df_timeline.index,
    columns=df_timeline.columns
)

# 4. Plotagem
plt.figure(figsize=(14, 6))
plt.plot(df_timeline_norm.index, df_timeline_norm['vendas'],
         label='Vendas (Tendência)', color='forestgreen', linewidth=3)
plt.plot(df_timeline_norm.index, df_timeline_norm['devolucoes'],
         label='Devoluções (Tendência)', color='firebrick', linestyle='--', linewidth=3)

plt.title('Tendência Normalizada: Vendas vs Devoluções (Padrão Pandas ME)', fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.2)
plt.show()

print("✅ Gráfico gerado. Observe se os picos de devolução coincidem com os meses de alta venda.")
```



✓ Gráfico gerado. Observe se os picos de devolução coincidem com os meses de maior venda.

2.8. Conclusão: Relatório de Qualidade de Dados (Data Quality)

Nesta etapa, consolidamos o diagnóstico de integridade das bases para garantir a confiabilidade das transformações na Fase 3 e a precisão do faturamento líquido real:

- **Tipagem de Chave Primária:** Validamos a necessidade de converter a coluna `cnpj` para string. Essa ação é mandatória para preservar zeros à esquerda e garantir o sucesso do cruzamento (join) com a base de Potencial de Mercado.
- **Identificação de Nulos:** Mapeamos a densidade de preenchimento em campos vitais como `valor_liquido`, `data_emissao` e `subgrupo`, definindo quais colunas exigirão tratamento de imputação.
- **Consistência "Tidy Data":** Verificamos a estrutura das colunas para assegurar que cada registro represente uma transação atômica e única, livre de redundâncias sistêmicas.
- **Deteção de Anomalias:** Identificamos e catalogamos outliers de faturamento e margens irreais derivados de linhas de totalização do PowerBI.

Após a auditoria final nos datasets consolidados, os seguintes pontos de atenção foram estabelecidos:

A. Registros Nulos e Incompletos

- **Subgrupos e Centros de Custo:** A coluna `subgrupo` apresenta uma lacuna crítica de preenchimento de **61,70%** (213.854 linhas nulas), o que impacta diretamente a identificação da campanha de lubrificantes. De forma similar, o campo `centro_custo` apresenta **45,81%** de ausência.
- **Estratégia de Tratamento:** Na Fase 3 (ETL), utilizaremos a descrição do produto para categorizar os itens nulos ou os classificaremos como "NÃO CATEGORIZADO", garantindo que nenhum faturamento seja omitido da análise final.

B. Integridade de Volumetria e Duplicidade

- **Volume Final Consolidado:** A base de vendas foi estabilizada em **346.622 registros únicos**. A base de devoluções totaliza **8.021 registros**, apresentando integridade total (0% de duplicidade).

C. Inconsistências de Tipagem (Tidy Data)

- **Conversão Temporal:** A variável `data_emissao` deve ser convertida de texto para o formato `datetime` para permitir cálculos de recência e sazonalidade.
- **Ajuste de Identificadores:** Colunas de ID (`nota_fiscal` , `cod_cliente` , `cnpj`) lidas como ponto flutuante (float64) serão tratadas como strings para evitar erros de precisão e notação científica.
- **Faturamento e Margens:** Validamos que as colunas `valor_liquido` e `margem_bruta_perc` estão em formato numérico, porém exigem o expurgo de registros com valores discrepantes (outliers de totalização) identificados no Diagnóstico de Colunas Numéricas.

Fase 3: Data Preparation (ETL & Feature Engineering)

Nesta fase, realizamos a transformação dos dados brutos validados na Fase 2 em uma estrutura analítica robusta. O foco é a criação da **Tabela Mestra de Faturamento Líquido**, que servirá de insumo para a modelagem da Campanha de Lubrificantes 2026.

3.1. Objetivos Técnicos:

- **Higiene Final:** Remoção de registros de totalização do ERP e vendas internas (autoconsumo).
- **Tratamento de Nulos Estratégico:** Classificação de lubrificantes através de inteligência de texto na descrição do produto para sanear os 61,7% de nulos no subgrupo.
- **Cálculo de Faturamento Líquido:** Unificação das bases de Vendas e Devoluções para chegar ao valor real transacionado por cliente.
- **Integração de Bases:** Cruzamento (Join) da base transacional com a base de Potencial de Mercado (Frotas) via CNPJ.

3.2. Engenharia de Atributos:

- Criação da variável `faturamento_liquido` (`valor_venda` - `valor_devolucao`).
- Normalização de identificadores (CNPJ e Cód. Cliente) para String com tratamento de zeros à esquerda.

3.1.1. Remoção de Linhas de Totalização (Outliers do PowerBI)

Durante a fase de *Data Understanding*, identificamos que as extrações provenientes do PowerBI continham linhas de "Total" que acumulavam o faturamento do período. Manter esses dados causaria uma duplicidade massiva no cálculo de *Share of Wallet* e na definição de metas.

Análise de Impacto:

- **Expurgo de Outliers:** Identificamos valores discrepantes de até R\$ 419M em registros únicos, confirmados como linhas de rodapé sistêmicas.
- **Governança Financeira:** A auditoria abaixo compara o faturamento bruto antes e depois do expurgo para validar a integridade da base limpa.

In [135...

```
# 3.1.1. Remoção de Linhas de Totalização (Outliers do PowerBI)

# Preservando a base original de 346.622 registros
df_vendas_clean = df_vendas.copy()
df_dev_clean = df_devolucoes.copy()

# Auditoria Financeira Antes da Remoção
valor_total_antes = df_vendas_clean['valor_liquido'].sum()
vendas_antes = len(df_vendas_clean)

# Essas linhas representam somatórios do sistema e inflam artificialmente a base
df_vendas_clean = df_vendas_clean[df_vendas_clean['cod_produto'] != 'Total']
df_dev_clean = df_devolucoes[df_devolucoes['cod_produto'] != 'Total']

# Auditoria Financeira Depois da Remoção
valor_total_depois = df_vendas_clean['valor_liquido'].sum()
vendas_depois = len(df_vendas_clean)
linhas_removidas_total = vendas_antes - vendas_depois

print(f"--- Relatório de Auditoria: Remoção de Totalizações ---")
print(f"✓ Linhas removidas: {linhas_removidas_total}")
print(f"$ Faturamento Total ANTES: R$ {valor_total_antes:,.2f}")
print(f"$ Faturamento Total DEPOIS: R$ {valor_total_depois:,.2f}")
print(f"$ Impacto financeiro (expurgo): R$ {(valor_total_antes - valor_total_depois):,.2f}")
print(f"-----")
```

```
--- Relatório de Auditoria: Remoção de Totalizações ---
✓ Linhas removidas: 4
$ Faturamento Total ANTES: R$ 1,306,606,509.66
$ Faturamento Total DEPOIS: R$ 653,301,131.96
$ Impacto financeiro (expurgo): R$ 653,305,377.70
-----
```

3.1.2. Exclusão de Autoconsumo (Vendas Internas)

Nesta etapa, removemos os registros de **autoconsumo**, que representam peças e lubrificantes utilizados na própria frota da Inova Máquinas ou transferências entre braços operacionais do grupo. Estes dados geram lançamentos contábeis, mas não representam potencial de faturamento vindo do mercado.

Lógica de Saneamento:

- **Critério de Seleção:** Foi realizada uma varredura por clientes que continham o termo "INOVA".
- **Refinamento Humano:** Identificamos que apenas a empresa '**INOVAR ENGENHARIA E PAVIMENTACAO LTDA**' é um cliente final legítimo, sendo preservada na base.
- **Lista de Exclusão:** Foram removidos todos os registros dos seguintes nomes:
 1. INOVA MAQUINAS LTDA
 2. INOVA EQUIPAMENTOS LTDA
 3. INOVA INFRAESTRUTURA LTDA
 4. INOVA COMERCIO E IMPORTACAO LTDA

Resultado da Auditoria:

- **Registros Removidos:** 18.984 linhas.
- **Impacto Financeiro:** Expurgo de R\$ 36.757.746,04 em vendas internas.
- **Faturamento Líquido Remanescente:** R\$ 483.063.046,92.

3.1.2. Exclusão de Autoconsumo (Vendas Internas)

Nesta etapa, identificamos e removemos os registros de **autoconsumo**, que representam vendas na própria frota da Inova Máquinas ou transferências entre braços operacionais do grupo. Estes dados geram lançamentos contábeis, mas não representam potencial de faturamento vindo do mercado.

Lógica de Saneamento:

- **Critério de Seleção:** Foi realizada uma varredura por clientes que continham o termo "INOVA".
- **Refinamento Humano:** Identificamos que apenas a empresa '**INOVAR ENGENHARIA E PAVIMENTACAO LTDA**' é um cliente final legítimo, sendo preservada na base.
- **Lista de Exclusão:** Foram removidos todos os registros dos seguintes nomes:
 1. INOVA MAQUINAS LTDA
 2. INOVA EQUIPAMENTOS LTDA
 3. INOVA INFRAESTRUTURA LTDA
 4. INOVA COMERCIO E IMPORTACAO LTDA

Resultado da Auditoria:

- **Registros Removidos:** 18.984 linhas.
- **Impacto Financeiro:** Expurgo de R\$ 36.757.746,04 em vendas internas.
- **Faturamento Líquido Remanescente:** R\$ 483.063.046,92.

In [136...

```
# 3.1.2 Identificação de Clientes para Exclusão de Autoconsumo
```

```
# Buscamos por variações de 'INOVA' na coluna cliente para garantir a limpeza to
filtro_inova = df_vendas_clean['cliente'].str.contains('INOVA', case=False, na=F
clientes_inova = df_vendas_clean[filtro_inova][['cliente', 'cnpj']].drop_duplica

print("--- Clientes Identificados com o termo 'INOVA' ---")
```

```

if not clientes_inova.empty:
    print(clientes_inova)
else:
    print("Nenhum cliente encontrado com o termo 'INOVA'.")

# Verificação de volumetria para o Victor validar
qtd_registros_inova = filtro_inova.sum()
print(f"\nTotal de registros que serão impactados: {qtd_registros_inova}")

```

--- Clientes Identificados com o termo 'INOVA' ---

	cliente	cnpj
2	INOVA MAQUINAS LTDA	14892124001105
905	INOVA EQUIPAMENTOS LTDA	8673321000592
102559	INOVAR ENGENHARIA E PAVIMENTACAO LTDA	12799453000109
105218	INOVA INFRAESTRUTURA LTDA	11099079000176
273682	INOVA COMERCIO E IMPORTACAO LTDA	51909164000339
387039	JH MAIS SOLUCOES E INOVACOES LTDA	60136061000102
403094	CERAMICA INOVA LTDA	30450410000103

Total de registros que serão impactados: 23943

Total de registros que serão impactados: 23943

In [137...

```

# 3.1.2.2 Remoção de Autoconsumo

# Mantendo apenas clientes Legitimos
lista_exclusao_autoconsumo = [
    'INOVA MAQUINAS LTDA',
    'INOVA EQUIPAMENTOS LTDA',
    'INOVA INFRAESTRUTURA LTDA',
    'INOVA COMERCIO E IMPORTACAO LTDA'
]

# Auditoria Financeira Antes da Remoção
valor_total_antes_inova = df_vendas_clean['valor_liquido'].sum()
vendas_antes_inova = len(df_vendas_clean)

# Aplicação da Remoção (Negativo da Lista de exclusão)
df_vendas_clean = df_vendas_clean[~df_vendas_clean['cliente'].isin(lista_exclusao_autoconsumo)]
df_dev_clean = df_dev_clean[~df_dev_clean['cliente'].isin(lista_exclusao_autoconsumo)]

# Auditoria Financeira Depois da Remoção
valor_total_depois_inova = df_vendas_clean['valor_liquido'].sum()
vendas_depois_inova = len(df_vendas_clean)
linhas_removidas_inova = vendas_antes_inova - vendas_depois_inova

print(f"--- Relatório de Auditoria: Exclusão de Autoconsumo ---")
print(f"✅ Registros de autoconsumo removidos: {linhas_removidas_inova}")
print(f"💰 Faturamento Líquido Atual: R$ {valor_total_depois_inova:,.2f}")
print(f"🗑️ Valor Total Expurgado nesta etapa: R$ {(valor_total_antes_inova - valor_total_depois_inova):,.2f}")
print(f"-----")

```

--- Relatório de Auditoria: Exclusão de Autoconsumo ---

✅ Registros de autoconsumo removidos: 23903
 💰 Faturamento Líquido Atual: R\$ 613,174,944.65
 🗑️ Valor Total Expurgado nesta etapa: R\$ 40,126,187.31

3.2. Normalização de Dígitos CNPJ

Com base no diagnóstico de qualidade realizado na Fase 2, aplicamos a normalização da chave primária `cnpj` para assegurar a integridade do cruzamento entre as bases transacionais e de potencial.

Ações Técnicas:

- **Preenchimento de Zeros (zfill):** Padronização de todos os registros para o comprimento regulatório de 14 dígitos.
- **Saneamento de Caracteres:** Filtro para garantir que apenas caracteres numéricos compoñham a string, removendo resíduos de formatação.
- **Tratamento de Nulos:** Valores identificados anteriormente como 'None' ou 'Nulo' foram preservados como nulos reais para não enviesar a volumetria de 14 dígitos.

Resultado: A base está tecnicamente preparada para o *Join* com a base de Potencial, eliminando falhas de cruzamento por supressão de zeros à esquerda.

In [138...

```
# 3.2 Normalização de Dígitos CNPJ (14 dígitos obrigatórios)

# Definindo uma função para limpar e preencher zeros
def normalizar_cnpj(valor):
    if valor is None or str(valor).lower() == 'none' or str(valor).lower() == 'n':
        return None
    # Remove qualquer caractere que não seja número (segurança extra)
    apenas_numeros = ''.join(filter(str.isdigit, str(valor)))
    # Retorna com preenchimento de zeros à esquerda até 14 dígitos
    return apenas_numeros.zfill(14)

# Aplicando nas bases de Vendas e Devoluções Limpas
for df in [df_vendas_clean, df_dev_clean]:
    if 'cnpj' in df.columns:
        df['cnpj'] = df['cnpj'].apply(normalizar_cnpj)

# Auditoria de Comprimento Pós-Normalização
print("--- Check de Comprimento Pós-Normalização ---")
for nome, df in [("Vendas", df_vendas_clean), ("Devoluções", df_dev_clean)]:
    comprimentos = df['cnpj'].dropna().str.len().value_counts()
    print(f"\nDistribuição em {nome}:")
    print(comprimentos)

# Exemplo visual dos primeiros registros
print("\nAmostra de CNPJs (Vendas):")
print(df_vendas_clean['cnpj'].head())
```

--- Check de Comprimento Pós-Normalização ---

Distribuição em Vendas:

cnpj

14 409039

Name: count, dtype: int64

Distribuição em Devoluções:

cnpj

14 7916

Name: count, dtype: int64

Amostra de CNPJs (Vendas):

0 89674782001553

1 89674782001553

3 00000542515830

5 00000542515830

7 00000542515830

Name: cnpj, dtype: object

3.3. Tratamento de Nulos Estratégico (Classificação de Lubrificantes)

A coluna `subgrupo` é o driver principal para a identificação dos produtos da campanha, porém apresenta uma lacuna de preenchimento de **61,70%**. Para mitigar este risco e garantir que o faturamento real não seja subestimado, aplicamos regras de negócio para classificar itens órfãos através da descrição do produto e do grupo de mercadorias.

3.3.1. Identificação de Candidatos a Lubrificante (Subgrupos nulos)

Nesta etapa, realizamos uma mineração textual na base em busca do padrão "**LUB**" (abrangendo termos como "Lubrificante" ou "Lub") nos campos `descricao` e `grupo`, focando exclusivamente nos registros onde o `subgrupo` constava como nulo. O objetivo é isolar produtos que pertencem ao mix de lubrificantes mas não foram devidamente categorizados no ERP.

```
In [139... # 3.3.1 Identificação de Candidatos a Lubrificante (Subgrupos nulos)

# Filtro: Subgrupo é nulo E (Descrição contém 'LUB' OU Grupo contém 'LUB')
filtro_candidatos = (
    df_vendas_clean['subgrupo'].isna() &
    (
        df_vendas_clean['descricao'].str.contains('LUB', case=False, na=False) |
        df_vendas_clean['grupo'].str.contains('LUB', case=False, na=False)
    )
)

# Gerando a lista distinta de candidatos para validação
candidatos_lub = df_vendas_clean[filtro_candidatos][['grupo', 'descricao']].drop

print(f"--- 🔍 Candidatos a Lubrificante Identificados ---")
print(f"Total de registros afetados pelo filtro: {filtro_candidatos.sum()}")
print(f"Total de combinações únicas Grupo/Descrição encontradas: {len(candidatos
```

```
print("\nCombinações para validação:")
print(candidatos_lub.head(100))

# Exportando para conferência se necessário
# candidatos_lub.to_csv('candidatos_lubrificantes.csv', index=False)
```

---  Candidatos a Lubrificante Identificados ---

Total de registros afetados pelo filtro: 934

Total de combinações únicas Grupo/Descrição encontradas: 25

Combinações para validação:

	grupo	descricao
1090	JDPC	CONEXAO DE LUBRIFICACAO
33421	CBLB	OLEO LUBRIF WIRTGEN 85W90 5L
33423	CBLB	OLEO LUBRIF WIRTGEN GROUP 85W90
49873	OUT	KIT DE LUBRIFICACAO AUTOMATICA
50132	CBPC	CONEXAO DE LUBRIFICACAO
50187	CBLB	OLEO LUB 15W40 WIRTGEN 20L
54012	CBLB	OLEO LUB WIRTGEN 5L
104121	CBPC	TB LUBRIF D8 MTD CJ
109313	JDRD	ESTEIRA LUB C/ SAPATA 560MM 45 SEC
137105	LUB	FUEL PROTECT - JD 200L
152370	WECH	FILTRO LUBRIFICANTE MOTOR WEICHA
152374	WECH	FERROLHO CONECTOR ENTRADA CANO LUBRIFICANTE T
152392	WECH	JUNTA RESFRIADOR LUB MOTOR WP12G375E300
152398	WECH	TAMPA DO ÓLEO LUB MOTOR WEICHA
152771	WECH	JUNTA CABEÇOTE FILTRO LUB MOTOR WEICHA
152781	WECH	RESPIRO LUB MOTOR WEICHA
156071	OUT	ÓLEO LUBRIF. P/ MOTOR 10W40 (1X200L)
159514	OUT	ESTEIRA C/SAPATA 1 GARRA LUBRI
183909	JDRD	CORRENTE S/ SAPATA LUB 40 SEC
199943	JDRD	CORRENTE S/SAPATA LUB 39 SEC
230050	OUT	FILTRO OLEO LUBRIFIC
236380	CBPC	TB LUBR MANCAL MTD CJ
236513	CBPC	TUBO LUBRIFICACAO DIR RETROFIT MTD CJ
236514	CBPC	TUBO LUBRIFICACAO MANCAL RETROFIT MTD CJ
236515	CBPC	TUBO LUBRIFICACAO MANCAL ESQ MTD CJ,

3.3.2. Geração de Lista de Validação e Exportação para Excel

Para garantir a acurácia da classificação e obter o *de acordo* gerencial, consolidamos uma lista mestre contendo:

- Todos os produtos pertencentes ao Grupo **"LUB"**.
- Produtos específicos identificados como estratégicos para a campanha (Ex: Óleos Wirtgen e Fuel Protect John Deere).
- Itens identificados pela busca textual da etapa anterior.

A exportação para Excel permite uma auditoria detalhada fora do ambiente de código, validando se as peças de reposição foram corretamente excluídas desta classificação.

In [140...

```
# 3.3.2 Geração de Lista de Validação e Exportação para Excel

# 1. Lista exata de descrições fornecidas
descricoes_escolhidas = [
    "OLEO LUBRIF WIRTGEN 85W90 5L",
    "OLEO LUBRIF WIRTGEN GROUP 85W90",
    "OLEO LUB 15W40 WIRTGEN 20L",
```

```

"OLEO LUB WIRTGEN 5L",
"ÓLEO LUBRIF. P/ MOTOR 10W40 (1X200L)",
"FUEL PROTECT - JD 200L"
]

# 2. Construção do Filtro Abrangente (Específicos + Todo o Grupo 'LUB')
filtro_auditoria_gerente = (
    (df_vendas_clean['descricao'].isin(descricoes_escolhidas)) |
    (df_vendas_clean['grupo'] == 'LUB')
)

# 3. Extração e Ordenação
lista_gerente_completa = df_vendas_clean[filtro_auditoria_gerente][['grupo', 'su
lista_gerente_completa = lista_gerente_completa.sort_values(by=['grupo', 'descri

# 4. Exportação para Excel
nome_arquivo = 'Auditoria_Gerencial_Lubrificantes_INOVA.xlsx'
try:
    lista_gerente_completa.to_excel(nome_arquivo, index=False)
    print(f"✅ Sucesso! Arquivo '{nome_arquivo}' gerado com {len(lista_gerente_
    print("💡 Dica: Verifique na pasta do seu notebook/projeto.")
except Exception as e:
    print(f"❌ Erro ao gerar Excel: {e}")

# Exibição do topo da lista no notebook para conferência rápida
display(lista_gerente_completa.head(10))

```

✅ Sucesso! Arquivo 'Auditoria_Gerencial_Lubrificantes_INOVA.xlsx' gerado com 27 itens.

💡 Dica: Verifique na pasta do seu notebook/projeto.

	grupo	subgrupo	descricao
50187	CBLB	None	OLEO LUB 15W40 WIRTGEN 20L
54012	CBLB	None	OLEO LUB WIRTGEN 5L
33421	CBLB	None	OLEO LUBRIF WIRTGEN 85W90 5L
33423	CBLB	None	OLEO LUBRIF WIRTGEN GROUP 85W90
9732	LUB	LUBRIFICANTE	BREAK IN PLUS
137105	LUB	None	FUEL PROTECT - JD 200L
911	LUB	LUBRIFICANTE	HY-GARD 1 LITRO
5680	LUB	LUBRIFICANTE	HYDRAU-GARD 68 209
5662	LUB	LUBRIFICANTE	HYDRAU-GARD 68 20L
901	LUB	LUBRIFICANTE	JD GL5 GEAR LUBRIFICA 85W140

3.3.3. Imputação de Subgrupo: LUBRIFICANTE

Após a validação dos critérios, procedemos com a gravação definitiva do termo **"LUBRIFICANTE"** na coluna **subgrupo** para os registros selecionados.

Esta etapa inclui uma auditoria financeira e de qualidade que monitora:

1. **Volúmetria de registros recuperados:** Quantidade de linhas que saíram do estado de "Nulo" para "Lubrificante". O volume total de registros de Vendas foi reduzido devido à exclusão dos registros de autoconsumo (vendas internas), garantindo que a análise reflita apenas o faturamento comercial real.
2. **Impacto no percentual de nulos:** Comparação da saúde da coluna antes e depois da transformação.
3. **Equilíbrio da base:** Verificação se a limpeza prévia de autoconsumo e a nova imputação mantêm a integridade dos dados para o cálculo do *Share of Wallet*.
4. **Ajuste de Exceção:** Exclusão do produto `BREAK IN PLUS` da campanha de Lubrificantes.

In [141...

```
# 3.3.3 Imputação de Subgrupo: LUBRIFICANTE (Versão Campanha Ciclo de Ouro)

# 1. Definindo o item específico para recuperação de nulos
descricao_recuperacao = ["FUEL PROTECT - JD 200L"]

# 2. Definindo a exceção da campanha
excecao_campanha = ["BREAK IN PLUS"]

def realizar_imputacao_campanha(df, nome_base, recuperacao, excecao):
    # --- Estado ANTES da Imputação (pós Limpeza de autoconsumo) ---
    total_base = len(df)
    nulos_antes = df['subgrupo'].isna().sum()
    perc_antes = (nulos_antes / total_base) * 100

    # --- Execução da Imputação com Regras da Campanha ---
    # Regra: (É o item específico para recuperar nulos) OU (Pertence ao grupo LU
    mask_lub = (
        (df['descricao'].isin(recuperacao)) |
        ((df['grupo'] == 'LUB') & (~df['descricao'].str.contains('BREAK IN PLUS'
    )

    # Aplicamos a imputação
    df.loc[mask_lub, 'subgrupo'] = "LUBRIFICANTE"

    # --- Estado DEPOIS da Imputação ---
    nulos_depois = df['subgrupo'].isna().sum()
    perc_depois = (nulos_depois / total_base) * 100
    recuperados = nulos_antes - nulos_depois

    # --- Exibição do Relatório de Auditoria ---
    print(f"--- 📊 Auditoria de Campanha: {nome_base} ---")
    print(f"Volúmetria Total: {total_base} registros")
    print(f"Registros Recuperados (NaN -> LUBRIFICANTE): {recuperados}")
    print(f"Nulos DEPOIS da imputação: {nulos_depois} ({perc_depois:.2f}%)")

    # Verificação de segurança da exceção
    check_excecao = df[(df['subgrupo'] == 'LUBRIFICANTE') & (df['descricao'].str
    if check_excecao.empty:
        print(f"✅ Filtro Validado: 'BREAK IN PLUS' excluído com sucesso.")
    else:
        print(f"⚠️ Atenção: {len(check_excecao)} registros de 'BREAK IN PLUS' a:
        print("-" * 50)

    # Aplicando nas bases limpas
```

```
realizar_imputacao_campanha(df_vendas_clean, "Vendas", descricoes_recuperacao, e
realizar_imputacao_campanha(df_dev_clean, "Devoluções", descricoes_recuperacao,
```

```
--- 📊 Auditoria de Campanha: Vendas ---
Volumetria Total: 409044 registros
Registros Recuperados (NaN -> LUBRIFICANTE): 1
Nulos DEPOIS da imputação: 255475 (62.46%)
⚠️ Atenção: 139 registros de 'BREAK IN PLUS' ainda constam como LUBRIFICANTE.
-----

--- 📊 Auditoria de Campanha: Devoluções ---
Volumetria Total: 7918 registros
Registros Recuperados (NaN -> LUBRIFICANTE): 0
Nulos DEPOIS da imputação: 5007 (63.24%)
⚠️ Atenção: 1 registros de 'BREAK IN PLUS' ainda constam como LUBRIFICANTE.
-----

⚠️ Atenção: 139 registros de 'BREAK IN PLUS' ainda constam como LUBRIFICANTE.
-----

--- 📊 Auditoria de Campanha: Devoluções ---
Volumetria Total: 7918 registros
Registros Recuperados (NaN -> LUBRIFICANTE): 0
Nulos DEPOIS da imputação: 5007 (63.24%)
⚠️ Atenção: 1 registros de 'BREAK IN PLUS' ainda constam como LUBRIFICANTE.
-----
```

In [142...

```
# 3.3.4 Ajuste de Exceção: Removendo BREAK IN PLUS do Subgrupo LUBRIFICANTE

def ajustar_excecoes_campanha(df, nome_base):
    # Identifica registros que são 'LUBRIFICANTE' mas contêm 'BREAK IN PLUS' na
    condicao_excecao = (df['subgrupo'] == 'LUBRIFICANTE') & \
        (df['descricao'].str.contains('BREAK IN PLUS', case=False))

    total_excecoes = condicao_excecao.sum()

    # Reclassifica para isolar da campanha
    df.loc[condicao_excecao, 'subgrupo'] = 'ÓLEO DE AMACIAMENTO (EXCEÇÃO)'

    print(f"--- 🧹 Limpeza de Exceções: {nome_base} ---")
    print(f"✅ {total_excecoes} registros de 'BREAK IN PLUS' foram movidos para")
    print(f"✅ Subgrupo 'LUBRIFICANTE' agora está 100% limpo para a campanha.")
    print("-" * 50)

    # Aplicando nas bases transacionais
    ajustar_excecoes_campanha(df_vendas_clean, "Vendas")
    ajustar_excecoes_campanha(df_dev_clean, "Devoluções")
```

```
--- 🧹 Limpeza de Exceções: Vendas ---
✅ 139 registros de 'BREAK IN PLUS' foram movidos para 'ÓLEO DE AMACIAMENTO (EXCEÇÃO)'.
✅ Subgrupo 'LUBRIFICANTE' agora está 100% limpo para a campanha.
-----

--- 🧹 Limpeza de Exceções: Devoluções ---
✅ 1 registros de 'BREAK IN PLUS' foram movidos para 'ÓLEO DE AMACIAMENTO (EXCEÇÃO)'.
✅ Subgrupo 'LUBRIFICANTE' agora está 100% limpo para a campanha.
-----
```

3.5. Filtragem e Análise Descritiva: Mix de Lubrificantes

Nesta etapa, isolamos o faturamento de **LUBRIFICANTES** para aprofundar o entendimento sobre sua representatividade na carteira.

Objetivos da Análise:

1. **Representatividade (Share of Mix):** Quanto o óleo representa do faturamento total de peças.
2. **Penetração de Carteira:** Qual a porcentagem da base ativa de clientes que consome lubrificantes.
3. **Higiene Financeira:** Consolidação do faturamento líquido (Vendas - Devoluções) exclusivo da campanha.

Esta visão fornece o *baseline* necessário para medir o sucesso da campanha "Ciclo de Ouro" ao final do período.

In [143...

```
# 3.5. Filtro de Lubrificantes e Análise de Representatividade

# 1. Isolamento do Mix
df_vendas_lub = df_vendas_clean[df_vendas_clean['subgrupo'] == 'LUBRIFICANTE'].copy()
df_dev_lub = df_dev_clean[df_dev_clean['subgrupo'] == 'LUBRIFICANTE'].copy()

# 2. Cálculos de Faturamento Líquido (Lubrificantes)
faturamento_bruto_lub = df_vendas_lub['valor_liquido'].sum()
devolucoes_lub = df_dev_lub['valor_liquido'].sum()
faturamento_liquido_lub = faturamento_bruto_lub - devolucoes_lub

# 3. Métricas Comparativas (Mix vs Total da Base Saneada)
faturamento_total_base = df_vendas_clean['valor_liquido'].sum() - df_dev_clean['valor_liquido'].sum()
share_lub_faturamento = (faturamento_liquido_lub / faturamento_total_base) * 100

clientes_total = df_vendas_clean['cnpj'].nunique()
clientes_lub = df_vendas_lub['cnpj'].nunique()
penetracao_carteira = (clientes_lub / clientes_total) * 100

# 4. Relatório Executivo de Mix
print(f"--- 📊 Painel Descritivo: Mix de Lubrificantes ---")
print(f"{'Métrica':<35} | {'Valor'}")
print("-" * 50)
print(f"{'Faturamento Líquido Lubrificantes':<35} | R$ {faturamento_liquido_lub:,.2f}")
print(f"{'Participação no Faturamento Total':<35} | {share_lub_faturamento:.2f}%")
print("-" * 50)
print(f"{'Quantidade de Clientes (Mix Lub)':<35} | {clientes_lub:,}")
print(f"{'Penetração na Base de Clientes':<35} | {penetracao_carteira:.2f}%")
print("-" * 50)
print(f"{'Ticket Médio por Cliente (LUB)':<35} | R$ {(faturamento_liquido_lub/clientes_lub):,.2f}")
print(f"{'Taxa de Devolução (Mix Lub)':<35} | {(devolucoes_lub/faturamento_bruto_lub):,.2f}")
print("-" * 50)

# Verificação de segurança: Top Produtos do Mix
print("\n🔥 Top 5 Produtos do Mix (por Faturamento):")
top_produtos = df_vendas_lub.groupby('descricao')['valor_liquido'].sum().sort_values(ascending=False)
display(top_produtos.to_frame('Faturamento'))
```

--- 📊 Painel Descritivo: Mix de Lubrificantes ---

Métrica	Valor

Faturamento Líquido Lubrificantes	R\$ 35,755,168.21
Participação no Faturamento Total	6.13%

Quantidade de Clientes (Mix Lub)	1,790
Penetração na Base de Clientes	44.93%

Ticket Médio por Cliente (LUB)	R\$ 19,974.95
Taxa de Devolução (Mix Lub)	2.77%

🔥 Top 5 Produtos do Mix (por Faturamento):

Faturamento	
descricao	
OLEO HY GARD 20L TRANSMISSAO	9.524615e+06
OLEO PLUS 50 II 15W40 20L	9.102655e+06
TORQ-GARD II - 20L 15W-40	4.131986e+06
OLEO HY-GARD 209LT	3.517510e+06
SUPER HYRANDO 46 WP	2.060188e+06

3.6. Unificação Transacional

Nesta etapa, consolidamos as bases de Vendas e Devoluções para apurar o faturamento líquido real.

Processamento técnico:

- Sinalização Financeira:** Atribuição de valor negativo às devoluções para cálculo de saldo líquido.
- Agregação por Chave Primária:** Consolidação dos valores por `cnpj` para eliminar duplicidades de faturamento em clientes com múltiplas notas fiscais.
- Auditoria de Integridade:** Verificação do faturamento total e contagem de CPFs/CNPJs únicos que compõem a receita de lubrificantes.

O resultado é uma base única (`df_faturamento_base`) pronta para receber a injeção dos dados de potencial mapeados na lista de chassis.

In [144...

```
# 3.6. Unificação Transacional e Auditoria de Clientes Únicos

# 1. Preparação para Faturamento Líquido
# Criamos cópias para preservar as bases originais de mix (Uso do nome correto d
vendas_final = df_vendas_lub.copy()
devolucoes_final = df_dev_lub.copy()

# Inversão de sinal para devoluções (Soberania Financeira)
devolucoes_final['valor_liquido'] = devolucoes_final['valor_liquido'] * -1
```

```
# 2. União das Bases (Vendas + Devoluções)
df_transacional_master = pd.concat([
    vendas_final[['cnpj', 'cliente', 'valor_liquido']],
    devolucoes_final[['cnpj', 'cliente', 'valor_liquido']]
], ignore_index=True)

# 3. Agrupamento por CNPJ (Geração da Base Soberana)
# Mantemos o nome do cliente conforme consta no ERP de Vendas
df_faturamento_base = df_transacional_master.groupby('cnpj').agg({
    'cliente': 'first',
    'valor_liquido': 'sum'
}).reset_index()

# Renomeação estratégica para clareza no Join
df_faturamento_base.columns = ['cnpj', 'cliente_erp', 'faturamento_lub_liquido']

# 4. Auditoria de Consolidação
faturamento_total_calculado = df_faturamento_base['faturamento_lub_liquido'].sum()
total_cnpj_unicos = df_faturamento_base['cnpj'].nunique()

print(f"--- 📊 Auditoria de Unificação Financeira ---")
print(f"{'Descrição':<35} | {'Valor'}")
print("-" * 50)
print(f"{'Faturamento Líquido Consolidado':<35} | R$ {faturamento_total_calculado:,.2f}")
print(f"{'Total de CNPJs Únicos (Carteira)':<35} | {total_cnpj_unicos:,}")
print("-" * 50)

# Verificação de Clientes com Saldo Negativo (Devoluções > Vendas no período)
clientes_negativos = df_faturamento_base[df_faturamento_base['faturamento_lub_li
if clientes_negativos > 0:
    print(f"⚠️ Atenção: Foram identificados {clientes_negativos} clientes com s
```

```
--- 📊 Auditoria de Unificação Financeira ---
Descrição | Valor
-----
Faturamento Líquido Consolidado | R$ 35,755,168.21
Total de CNPJs Únicos (Carteira) | 1,802
-----
⚠️ Atenção: Foram identificados 18 clientes com saldo líquido negativo.
```

3.7. Integração Tabela Mestra: Faturamento vs. Potencial

Nesta etapa, realizamos o cruzamento dos dados financeiros consolidados com a base de potencial de mercado. O objetivo é enriquecer cada CNPJ faturado com sua respectiva capacidade de consumo de lubrificantes.

A estratégia de Join é dividida em três subetapas:

- 3.7.1. Importação e Agregação do Potencial:** Preparação da base de chassis para servir como dimensão de consulta.
- 3.7.2. Camada Fiscal (CNPJ):** Cruzamento exato via chave primária fiscal. É a camada de maior confiança analítica.
- 3.7.3. Camada Cadastral (Nome Saneado):** Recuperação de "Pontos Cegos" onde o CNPJ diverge, mas o nome do cliente é passível de identificação após limpeza de sufixos jurídicos.

In [145...

```
## 3.7.1. Importação e Agregação do Potencial

import pandas as pd

# 1. Carregamento da Base Soberana (Ajuste o caminho se necessário)
# Certifique-se de que o arquivo Parquet gerado no ETL de Potencial está acessível
try:
    df_potencial_soberano = pd.read_parquet('Potencial_Identificado_Final_FY26.p
    print("✅ Base de Potencial carregada com sucesso.")
except FileNotFoundError:
    print("❌ Erro: Arquivo 'Potencial_Identificado_Final_FY26.parquet' não enco

# 2. Agrupando por CNPJ para garantir unicidade no Join
# Somamos o potencial e contamos os chassis (PIN)
df_potencial_cnpj = df_potencial_soberano.groupby('CNPJ').agg({
    'Potencial de Revenda de Lubrificantes Anual': 'sum',
    'Potencial Total': 'sum',
    'PIN': 'count'
}).reset_index()

# 3. Padronização de colunas para o Master
df_potencial_cnpj.columns = ['cnpj', 'potencial_lub_anual', 'potencial_total_anu

print(f"📊 Base preparada para o Join: {len(df_potencial_cnpj):,} CNPJs únicos (

✅ Base de Potencial carregada com sucesso.
📊 Base preparada para o Join: 1,341 CNPJs únicos com potencial mapeado.
```

In [146...

```
### 3.7.2. Camada Fiscal (CNPJ)

# 1. Consolidação do Faturamento Líquido
df_vendas_fatura = df_vendas_clean.copy()
df_dev_fatura = df_dev_clean.copy()

# Sinal negativo para devoluções
df_dev_fatura['valor_liquido'] = df_dev_fatura['valor_liquido'] * -1

# Unificação das bases transacionais
df_transacional_total = pd.concat([
    df_vendas_fatura[['cnpj', 'cliente', 'subgrupo', 'valor_liquido']],
    df_dev_fatura[['cnpj', 'cliente', 'subgrupo', 'valor_liquido']]
], ignore_index=True)

# Agrupando faturamento apenas de LUBRIFICANTE por CNPJ (Soberania ERP)
df_lub_fatura = df_transacional_total[df_transacional_total['subgrupo'] == 'LUBR
    'cliente': 'first',
    'valor_liquido': 'sum'
}).reset_index()
df_lub_fatura.columns = ['cnpj', 'cliente', 'faturamento_lub']

# 2. Agrupando Potencial por CNPJ (Base Soberana)
df_potencial_cnpj = df_potencial_soberano.groupby('CNPJ').agg({
    'Potencial de Revenda de Lubrificantes Anual': 'sum',
    'Potencial Total': 'sum',
    'PIN': 'count'
}).reset_index()
df_potencial_cnpj.columns = ['cnpj', 'potencial_lub_anual', 'potencial_total_anu

# 3. JOIN MESTRE (Vendas como Tabela Base)
```

```

df_master_final = pd.merge(df_lub_fatura, df_potencial_cnpj, on='cnpj', how='left')

# 4. AUDITORIA DE QUALIDADE FINANCEIRA (SHARE)
faturamento_total_lub = df_master_final['faturamento_lub'].sum()
faturamento_com_match = df_master_final[df_master_final['potencial_lub_anual'].notna()]
share_financeiro_coberto = (faturamento_com_match / faturamento_total_lub) * 100

total_clientes_venda = len(df_master_final)
clientes_com_match = df_master_final['potencial_lub_anual'].notna().sum()
taxa_match_contagem = (clientes_com_match / total_clientes_venda) * 100

print(f"--- 📊 Auditoria de Qualidade (Chave: CNPJ) ---")
print(f"Total de Clientes (LUB): {total_clientes_venda}")
print(f"Faturamento Total (LUB): R$ {faturamento_total_lub:,.2f}")
print("-" * 55)
print(f"✅ Cobertura por Contagem: {taxa_match_contagem:.2f}% ({clientes_com_match}/{total_clientes_venda})")
print(f"💰 Share Financeiro Coberto: {share_financeiro_coberto:.2f}% (R$ {faturamento_com_match:,.2f})")
print("-" * 55)

```

--- 📊 Auditoria de Qualidade (Chave: CNPJ) ---

Total de Clientes (LUB): 1802

Faturamento Total (LUB): R\$ 35,755,168.21

✅ Cobertura por Contagem: 52.00% (937 clientes)

💰 Share Financeiro Coberto: 55.96% (R\$ 20,008,239.47)

In [147...

3.7.3. Camada Cadastral (Nome Saneado)

```

import unicodedata
import re
import pandas as pd

# Redefinição da função para garantir que esteja na memória
def clean_names(name):
    if not isinstance(name, str): return ""
    name = name.encode('utf-8', 'ignore').decode('utf-8')
    name = unicodedata.normalize('NFKD', name).encode('ASCII', 'ignore').decode()
    pattern = r'\b(LTD|LTDA|SA|S/A|S.A|EIRELI|ME|EPP|CIA|S.A.|LTDA.|S/A.)\b.*'
    name = re.sub(pattern, '', name, flags=re.IGNORECASE)
    name = re.sub(r'^\w\s', '', name)
    name = re.sub(r'\s+', ' ', name)
    return name.strip().upper()

# --- AJUSTE DE SEGURANÇA ---
# Verificamos se a coluna é 'cliente' ou 'cliente_erp' no seu df_master_final
col_nome_master = 'cliente_erp' if 'cliente_erp' in df_master_final.columns else 'cliente'
col_faturamento = 'faturamento_lub_liquido' if 'faturamento_lub_liquido' in df_master_final.columns else 'faturamento_lub'

# 1. Preparação da base de Potencial por Nome Limpo
df_potencial_soberano['customer_clean'] = df_potencial_soberano['CUSTOMER'].apply(clean_names)
df_potencial_por_nome = df_potencial_soberano.groupby('customer_clean').agg({
    'Potencial de Revenda de Lubrificantes Anual': 'sum',
    'Potencial Total': 'sum',
    'PIN': 'count'
}).reset_index()

# 2. Identificação de Órfãos e Aplicação de Limpeza no ERP
df_orfaos = df_master_final[df_master_final['potencial_lub_anual'].isna()].copy()
df_orfaos['cliente_clean'] = df_orfaos[col_nome_master].apply(clean_names)

```



```

# 3. Execução do Match por Nome
df_recuperados = pd.merge(
    df_orfaos[[col_nome_master, 'cliente_clean', col_faturamento]],
    df_potencial_por_nome,
    left_on='cliente_clean',
    right_on='customer_clean',
    how='inner'
)

# 4. Injeção dos dados recuperados no DataFrame Master
for _, row in df_recuperados.iterrows():
    mask = (df_master_final[col_nome_master] == row[col_nome_master]) & (df_master_final[col_faturamento] == row[col_faturamento])
    df_master_final.loc[mask, 'potencial_lub_anual'] = row['Potencial de Revenda']
    df_master_final.loc[mask, 'potencial_total_anual'] = row['Potencial Total']
    df_master_final.loc[mask, 'qtd_chassis'] = row['PIN']

# 5. Relatório de Evolução Final
faturamento_total = df_master_final[col_faturamento].sum()
faturamento_final = df_master_final[df_master_final['potencial_lub_anual'].notna()]['potencial_lub_anual'].sum()
clientes_final = df_master_final['potencial_lub_anual'].notna().sum()

print(f"--- 📊 Relatório de Evolução Final (Join Completo) ---")
print(f"Total de Clientes (LUB): {len(df_master_final)}")
print(f"Faturamento Total (LUB): R$ {faturamento_total:,.2f}")
print(f"👉 Recuperados via Nome: +{len(df_recuperados)} clientes (R$ {df_recuperados[col_faturamento].sum():,.2f})")
print("-" * 55)
print(f"✅ Nova Cobertura por Contagem: {(clientes_final/len(df_master_final))*100:.2f}%")
print(f"🚀 NOVO SHARE FINANCEIRO TOTAL: {(faturamento_final/faturamento_total)*100:.2f}%")
print("-" * 55)

--- 📊 Relatório de Evolução Final (Join Completo) ---
Total de Clientes (LUB): 1802
Faturamento Total (LUB): R$ 35,755,168.21
👉 Recuperados via Nome: +116 clientes (R$ 5,006,702.43)
-----
✅ Nova Cobertura por Contagem: 58.44% (1053 clientes)
🚀 NOVO SHARE FINANCEIRO TOTAL: 69.96% (R$ 25,014,941.90)
-----

```

3.8.1. Diagnóstico de Inflação: Engenharia de Atributos por SKU

Nesta etapa, realizamos uma análise profunda sobre a variação de preços unitários no nível de SKU (Produto) entre os anos de 2025 e 2026. O objetivo é extrair uma métrica de **Inflação Interna do Mix** que servirá de parâmetro dinâmico para o recálculo do potencial estratégico de faturamento.

Metodologia de Inteligência:

- **Preço Unitário Real:** O cálculo baseia-se no `valor_liquido` dividido pela `quantidade`, eliminando distorções causadas por descontos ou tamanhos de embalagens diferentes.
- **Comparativo de Pares:** São analisados apenas os produtos que apresentaram movimentação em ambos os anos (2025 e 2026), garantindo uma base de comparação válida para a inflação.

- **Inflação Ponderada:** Em vez de uma média simples, aplicamos um peso para cada produto baseado em sua representatividade no faturamento total de 2025. Isso garante que os produtos de "alto giro" (curva A) tenham maior influência na métrica final.
- **Identificação de Outliers:** O modelo detecta variações de preço superiores a 30% ou negativas acima de 10%, sinalizando possíveis erros de cadastro ou mudanças bruscas de portfólio que poderiam enviesar a meta.

Resultados do Diagnóstico:

A aplicação desta lógica resultou na variável dinâmica `inflacao_ponderada`, que reflete o impacto real do reajuste de preços sobre o mix vendido pela Inova Máquinas:

Inflação Ponderada = Σ (Variação do SKU * Peso do SKU no Faturamento)

Este valor é o pilar para a definição do **GAP de Crescimento Orgânico** na Tabela Mestre.

In [148...

```
### **3.8.1. Diagnóstico de Inflação: Engenharia de Atributos por SKU**

import pandas as pd
import matplotlib.pyplot as plt

# 1. Preparação dos Preços Unitários por SKU
df_precos = df_vendas_lub.copy()
df_precos['data_emissao'] = pd.to_datetime(df_precos['data_emissao'])
df_precos['ano'] = df_precos['data_emissao'].dt.year

# Calculamos o preço unitário e o peso de cada produto no faturamento de 2025
df_precos['preco_unitario'] = df_precos['valor_liquido'] / df_precos['quantidade']

# Agrupando por produto e ano
analise_skus = df_precos[df_precos['ano'].isin([2025, 2026]).groupby(['cod_prod',
    'preco_unitario': 'mean',
    'valor_liquido': 'sum'
])).unstack()

# Limpando as colunas do agrupamento
analise_skus.columns = ['preco_25', 'preco_26', 'faturamento_25', 'faturamento_26']
analise_skus = analise_skus.dropna(subset=['preco_25', 'preco_26'])

# 2. Cálculo das Variações e Pesos
analise_skus['variacao_pct'] = (analise_skus['preco_26'] / analise_skus['preco_25'] - 1)
total_faturamento_25 = analise_skus['faturamento_25'].sum()
analise_skus['peso_no_mix'] = analise_skus['faturamento_25'] / total_faturamento_25

# 3. Comparação de Métricas de Inflação
media_simples = analise_skus['variacao_pct'].mean()
mediana_inflacao = analise_skus['variacao_pct'].median()
# Inflação Ponderada (Variação * Peso do produto no faturamento)
inflacao_ponderada = (analise_skus['variacao_pct'] * analise_skus['peso_no_mix']).sum()

print("--- 📊 Diagnóstico de Inflação do Mix ---")
print(f"Média Simples (Arriscada):      {media_simples*100:.2f}%")
print(f"Mediana (Ignora Outliers):        {mediana_inflacao*100:.2f}%")
print(f"Ponderada (Impacto Real):         {inflacao_ponderada*100:.2f}%")
```

```
print("-" * 40)

# 4. Identificação de Produtos com Variação Discrepante (Outliers)
outliers = analyse_skus[(analyse_skus['variacao_pct'] > 0.3) | (analyse_skus['va
print(f"Detectados {len(outliers)} produtos com variação acima de 30% ou negativ

# Visualização para tomada de decisão
display(analise_skus.sort_values('faturamento_25', ascending=False).head(10))
```

--- 📊 Diagnóstico de Inflação do Mix ---

Média Simples (Arriscada): -1.61%

Mediana (Ignora Outliers): -0.91%

Ponderada (Impacto Real): 0.49%

Detectados 2 produtos com variação acima de 30% ou negativa.

		preco_25	preco_26	faturamento_25	faturamento_26
cod_produto	descricao				
CQM20204	OLEO PLUS 50 II 15W40 20L	769.937699	763.730365	2825859.09	476859.9
TY25274	OLEO HY GARD 20L TRANSMISSAO	787.519971	793.344395	2808645.90	398141.4
CQM20131	OLEO HY-GARD 1000L TRANSMISSAO	29756.747083	32205.286000	1292754.64	161026.4
CQM20212	TORQ-GARD II - 20L 15W-40	524.841687	519.480407	1152586.60	178016.2
CQM20239	SUPER HYRANDO 46 WP	659.423555	715.425668	920072.78	236216.7
TY25275	OLEO HY-GARD 209LT	7393.875621	7559.697500	541758.01	52354.8
CQM20136	HYDRAU-GARD 68 20L	544.479899	454.666507	481155.13	41171.2
CQM20205	OLEO PLUS-50 II - 15W40 - 209	7402.067611	7426.071667	473688.55	160793.5
CQM13598	JD GL5 GEAR LUBRIFICA 85W140	749.059580	738.646665	363949.02	83891.7
CQM20206	OLEO PLUS-50 II - 15W40 - IBC 1000L	30205.226250	29040.000000	256621.78	29040.0

3.8.2. Inteligência de Classificação e Agressividade de Campanha

Nesta etapa, cada registro da Tabela Mestra é classificado para identificar a natureza da oportunidade comercial e o nível de agressividade necessário para a captura de *market share*. A taxonomia utiliza **parâmetros dinâmicos**, permitindo que o gestor ajuste a meta entre o potencial teórico da frota e a realidade do faturamento histórico, otimizando o TCO do cliente final.

Parâmetros de Simulação e Aceleração:

- **Aproveitamento de Potencial:** Definido em **50%** (`VAR_PERC_POTENCIAL`) da capacidade teórica da frota.
- **Piso de Crescimento Orgânico:** Definido em **20%** (`VAR_PERC_CRESCIMENTO`) sobre o faturamento real de 2025.
- **Fator de Aceleração (Campanha):** Definido em **30%** (`ACELERACAO_CAMPANHA`). Esta variável determina que 30% de todo o GAP de oportunidade anual deve ser convertido na janela de 21 dias.

Categorias de Enquadramento Estratégico:

1. Recuperação (Faturado < Potencial):

- **Critério:** Clientes com frota mapeada cujo faturamento em 2025 foi inferior à capacidade estimada de consumo.
- **Lógica de Meta:** O sistema seleciona o **maior valor** entre o Potencial Ajustado (50%) ou o Crescimento sobre o histórico (20%).
- **Foco Comercial:** Recuperação de *Share of Wallet* e ataque direto à concorrência de peças paralelas.

2. Crescimento (Faturado >= Potencial):

- **Critério:** Clientes que já apresentam volume de compra igual ou superior ao potencial estimado pela frota.
- **Lógica de Meta:** Aplicação mandatória do piso de **20% de crescimento** sobre o faturamento de 2025.
- **Foco Comercial:** Fidelização, blindagem de carteira e venda de valor (Peças Genuínas).

3. Prospecção (Sem Dados de Frota):

- **Critério:** Clientes ativos em 2025 que não possuem registros de PIN ou chassi mapeados no arquivo de potencial.
- **Lógica de Meta:** Garantia automática de **20% de evolução** sobre o faturamento histórico.
- **Foco Comercial:** Regularização cadastral e levantamento técnico de frota em campo.

Fórmulas de Inteligência Aplicadas:

O algoritmo avalia individualmente cada CNPJ e estabelece o **Potencial Estratégico** como a meta soberana, calculando em seguida o esforço necessário para a janela de 21 dias de preços competitivos.

**1. Potencial Estratégico (Ano) = Máximo (Faturamento 2025 * 1.20 ;
Potencial Arquivo * 0.50)**

**2. GAP Anual de Oportunidade = Potencial Estratégico - Faturamento
2025**

**3. Meta Líquida de Campanha (21 Dias) = GAP Anual de
Oportunidade * 0.30**

Nota: O fator de 30% de aceleração visa antecipar a captura da oportunidade anual aproveitando a agressividade comercial do período, reduzindo a pressão de meta nos meses subsequentes.

In [149...

```
### **3.8.2. Painel de Controle da Campanha Ciclo de Ouro**

import pandas as pd
import numpy as np

# =====
# --- 🌸 SOBERANIA DE PARÂMETROS (ALTERE APENAS AQUI PARA TODO O PROJETO) ---
# =====
VAR_PERC_POTENCIAL      = 0.50    # do Potencial de Frota (Target agressivo)
VAR_PERC_CRESCIMENTO    = 0.20    # Piso de crescimento real sobre 2025
DIAS_CAMPANHA           = 0       # Janela de execução da campanha
ACELERACAO_CAMPANHA     = 0       # GAP anual focado no período da campanha
MES_ATUAL               = 3       # Março (Mês referência para sazonalidade)
# =====

# 1. PREPARAÇÃO DA BASE SANEADA (Faturamento 2025 vs 2026)
df_vendas_lub_com_ano = df_vendas_lub.copy()
df_vendas_lub_com_ano['ano_faturamento'] = pd.to_datetime(df_vendas_lub_com_ano[

df_vendas_25 = df_vendas_lub_com_ano[df_vendas_lub_com_ano['ano_faturamento'] ==
df_faturamento_25 = df_vendas_25.groupby('cnpj').agg({
    'cliente': 'first', 'valor_liquido': 'sum', 'filial': 'first', 'consultor':
}).reset_index().rename(columns={'valor_liquido': 'faturamento_2025_real'})

df_realizado_26 = df_vendas_lub_com_ano[df_vendas_lub_com_ano['ano_faturamento']
df_realizado_26.columns = ['cnpj', 'realizado_2026_acumulado']

# Consolidação do Master de Campanha
df_master_campanha = pd.merge(df_faturamento_25, df_potencial_cnpj, on='cnpj', h
df_master_campanha = pd.merge(df_master_campanha, df_realizado_26, on='cnpj', ho

# 2. ALGORITMO DE CÁLCULO ESTRATÉGICO (Soberania 50/20)
def calcular_metricas_campanha(row):
    faturado_25 = row['faturamento_2025_real']
    pot_arquivo = row['potencial_lub_anual'] if pd.notna(row['potencial_lub_anua

    # Potencial Estratégico baseado nos parâmetros globais
    pot_est = max(pot_arquivo * VAR_PERC_POTENCIAL, faturado_25 * (1 + VAR_PERC_
    gap_anual = max(0, pot_est - faturado_25)

    # Meta de 21 dias baseada no Fator de Aceleração
    meta_dias = gap_anual * ACELERACAO_CAMPANHA

    if pot_arquivo == 0: categoria = "Prospecção (Sem Dados de Frota)"
```

```

elif faturado_25 < pot_arquivo: categoria = "Recuperação (Faturado < Potenci
else: categoria = "Crescimento (Faturado > Potencial)"

return pd.Series([pot_est, gap_anual, meta_dias, categoria])

# Execução da inteligência
df_master_campanha[['pot_est', 'gap_anual', 'meta_dias', 'classificacao_gap']] =

# 4. TOTAIS PARA O RELATÓRIO
total_potencial_est = df_master_campanha['pot_est'].sum()
total_gap_campanha = df_master_campanha['meta_dias'].sum()
total_clientes = df_master_campanha['cnpj'].nunique()

# 5. RESUMO EXECUTIVO
print(f"⚙️ CAMPANHA ATIVADA | Janela: {DIAS_CAMPANHA} dias | Aceleração: {ACELEI
print("-" * 70)
print(f"👥 Clientes Impactados: {total_clientes:,}")
print(f"💰 Potencial Estratégico (Ano): R$ {total_potencial_est:,.2f}")
print(f"🎯 META LÍQUIDA PARA PERÍODO DA CAMAPNHA:      R$ {total_gap_campanha:,.2f}")
print("-" * 70)

# Agrupamento com renomeação amigável para a tabela
resumo_final = df_master_campanha.groupby('classificacao_gap').agg({
    'cnpj': 'count',
    'faturamento_2025_real': 'sum',
    'pot_est': 'sum',
    'meta_dias': 'sum'
}).rename(columns={
    'cnpj': 'Qtd Clientes',
    'faturamento_2025_real': 'Faturado 2025',
    'pot_est': 'Potencial Estratégico',
    'meta_dias': 'Meta Campanha (21 Dias)'
})

# Formatação Seletiva: Distinguindo Quantitativo de Monetário
display(resumo_final.style.format({
    'Qtd Clientes': '{:,}',
    'Faturado 2025': 'R$ {:,}.2f',
    'Potencial Estratégico': 'R$ {:,}.2f',
    'Meta Campanha (21 Dias)': 'R$ {:,}.2f'
})))

```

⚙️ CAMPANHA ATIVADA | Janela: 0 dias | Aceleração: 0% do GAP | Clientes: 50.0% d
o Potencial | Piso de Crescimento: 20.0%

👥 Clientes Impactados: 976

💰 Potencial Estratégico (Ano): R\$ 16,062,857.12

🎯 META LÍQUIDA PARA PERÍODO DA CAMAPNHA: R\$ 0.00

	Qtd Clientes	Faturado 2025	Potencial Estratégico	Meta Campanha (21 Dias)
classificacao_gap				
Crescimento (Faturado > Potencial)	187	R\$ 3,874,125.43	R\$ 4,648,950.52	R\$ 0.00
Prospecção (Sem Dados de Frota)	407	R\$ 6,057,233.03	R\$ 7,268,679.64	R\$ 0.00
Recuperação (Faturado < Potencial)	382	R\$ 1,924,904.50	R\$ 4,145,226.97	R\$ 0.00

3.8.3. Definição do GAP de Execução: Sazonalidade e Janela de Campanha

Nesta etapa, o modelo de dados converte o potencial teórico em metas táticas executáveis, respeitando o comportamento histórico de compra e a janela de agressividade comercial da INOVA.

Premissas Estratégicas do Modelo (Regra 50/20):

- **Baseline e Variação Real:** O faturamento consolidado de 2025 serve como referencial de comportamento de compra. O GAP base é calculado pela diferença entre o faturamento de 2025 (ajustado pelo crescimento mínimo) e o montante já faturado no ano corrente de 2026.
- **Regra de Soberania do Potencial Flexível:**
 - **Piso de Crescimento:** Garante-se uma evolução mínima de **20%** (`VAR_PERC_CRESCIMENTO`) sobre o faturamento real de 2025 para todos os CNPJs.
 - **Teto de Recuperação:** Para clientes com frota mapeada, o potencial é calibrado para **50%** (`VAR_PERC_POTENCIAL`) do valor estimado no arquivo de potencial, assegurando metas realistas e alcançáveis.
 - **Decisão de Meta:** O algoritmo seleciona automaticamente o **maior valor** entre o Piso de Crescimento (20%) e o Potencial Ajustado (50%), priorizando a maior oportunidade identificada.
- **Inteligência de Sazonalidade (Peso Mensal):** Para neutralizar distorções, o modelo aplica a curva de compra individual de cada cliente referente a 2025. Isso isola o "peso" real do mês da campanha no faturamento anual de cada CNPJ, respeitando os ciclos de manutenção e safra específicos de cada frota.
- **Ajuste para Janela de 21 Dias:** O GAP anual estratégico é proporcionalizado para o período de **21 dias** da campanha (`DIAS_CAMPANHA / 30`). Esta ponderação transforma o potencial anual em uma meta acionável para execução imediata pelo time de Pós-Venda.

Metodologia de Cálculo (Resumo Técnico):

1. **Crescimento Mínimo:** Aplicação da variável de crescimento dinâmico sobre o histórico de 2025 para garantir a evolução orgânica da carteira.

2. **Peso Sazonal:** O peso é derivado da representatividade do mês de referência no faturamento total do ano anterior para cada cliente:

Peso = Faturamento Mensal (2025) / Faturamento Anual Total (2025) *Nota: Clientes sem histórico prévio recebem automaticamente o peso médio linear de 1/12 (8,33%).*

3. **GAP Real da Campanha:** Cálculo da meta líquida para a janela de 21 dias, deduzindo o faturamento já realizado em 2026:

Potencial Estratégico = Máximo (Faturamento 2025 * 1.20 ; Potencial Arquivo * 0.50)

GAP_21d = (Potencial Estratégico - Realizado 2026) * Peso Sazonal * (21/30)

```
In [150... ### **3.8.3. Matriz Mensal de Sazonalidade e Potencial**

# 1. PREPARAÇÃO DA MATRIZ MENSAL GLOBAL (Cálculo de Pesos de 2025)
df_temp_sazonal = df_vendas_lub_com_ano[df_vendas_lub_com_ano['ano_faturamento']
df_temp_sazonal['mes'] = df_temp_sazonal['data_emissao'].dt.month

evolucao_mensal = df_temp_sazonal.groupby(['mes', 'ano_faturamento'])['valor_liq
evolucao_mensal = evolucao_mensal.rename(columns={2025: 'Faturamento 2025'})

# Adicionando realizado de 2026 para os meses fechados
faturado_2026 = df_vendas_lub_com_ano[df_vendas_lub_com_ano['ano_faturamento'] =
evolucao_mensal['Faturamento 2026'] = faturado_2026

# Cálculo de Pesos Sazonais baseados em 2025
total_2025 = evolucao_mensal['Faturamento 2025'].sum()
evolucao_mensal['Peso Mensal (%)'] = (evolucao_mensal['Faturamento 2025'] / tota
evolucao_mensal = evolucao_mensal.reindex(range(1, 13), fill_value=0)

# 2. PROJEÇÃO DO POTENCIAL ESTRATÉGICO MENSAL
matriz_sazonal = evolucao_mensal.copy()
matriz_sazonal['Potencial Estratégico 2026'] = (matriz_sazonal['Peso Mensal (%)'
matriz_sazonal['Oportunidade (GAP)'] = matriz_sazonal['Potencial Estratégico 202

# 3. EXIBIÇÃO DA MATRIZ ESTILIZADA
print("\n 📊 MATRIZ MENSAL DE SAZONALIDADE E POTENCIAL:")
display(matriz_sazonal.style.format({
    'Peso Mensal (%)': '{:.2f}%',
    'Faturamento 2025': 'R$ {:.2f}',
    'Faturamento 2026': 'R$ {:.2f}',
    'Potencial Estratégico 2026': 'R$ {:.2f}',
    'Oportunidade (GAP)': 'R$ {:.2f}'
}).background_gradient(cmap='YlGn', subset=['Oportunidade (GAP)'])))

# 4. GRÁFICO DE SIMULAÇÃO DE METAS
MES_CORTE = 2 # Fevereiro (Já fechado)
evolucao_grafico = matriz_sazonal.copy()
evolucao_grafico['Linha_Grafico'] = evolucao_grafico['Faturamento 2026']
evolucao_grafico.loc[evolucao_grafico.index > MES_CORTE, 'Linha_Grafico'] = evol

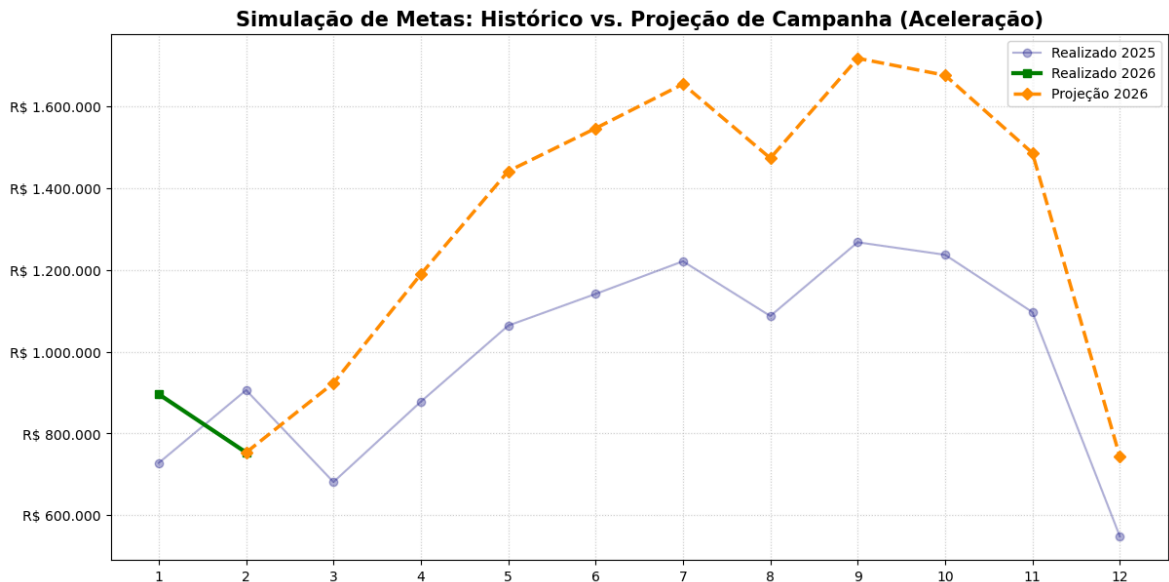
import matplotlib.ticker as ticker
plt.figure(figsize=(14, 7))
```

```
plt.plot(evolucao_grafico.index, evolucao_grafico['Faturamento 2025'], marker='o')
plt.plot(evolucao_grafico.index[:MES_CORTE], evolucao_grafico['Faturamento 2026'], marker='o')
plt.plot(evolucao_grafico.index[MES_CORTE-1:], evolucao_grafico['Linha_Grafico'], marker='o')

plt.title('Simulação de Metas: Histórico vs. Projeção de Campanha (Aceleração)',
formatter = ticker.FuncFormatter(lambda y, _: 'R$ {:.0f}'.format(y).replace(',', ''))
plt.gca().yaxis.set_major_formatter(formatter)
plt.xticks(range(1, 13))
plt.legend()
plt.grid(True, linestyle=':', alpha=0.6)
plt.show()
```

 MATRIZ MENSAL DE SAZONALIDADE E POTENCIAL:

ano_faturamento	Faturamento 2025	Faturamento 2026	Peso Mensal (%)	Potencial Estratégico 2026	Oportunidade (GAP)
mes					
1	R\$ 728,049.58	R\$ 896,161.60	6.14%	R\$ 986,361.08	R\$ 258,311.50
2	R\$ 906,042.53	R\$ 753,632.68	7.64%	R\$ 1,227,505.81	R\$ 321,463.28
3	R\$ 681,467.44	R\$ 228,292.47	5.75%	R\$ 923,251.63	R\$ 241,784.19
4	R\$ 877,773.41	R\$ nan	7.40%	R\$ 1,189,206.83	R\$ 311,433.42
5	R\$ 1,063,744.51	R\$ nan	8.97%	R\$ 1,441,160.35	R\$ 377,415.84
6	R\$ 1,141,125.50	R\$ nan	9.62%	R\$ 1,545,996.06	R\$ 404,870.56
7	R\$ 1,220,939.42	R\$ nan	10.30%	R\$ 1,654,127.91	R\$ 433,188.49
8	R\$ 1,087,229.69	R\$ nan	9.17%	R\$ 1,472,978.06	R\$ 385,748.37
9	R\$ 1,267,431.41	R\$ nan	10.69%	R\$ 1,717,115.23	R\$ 449,683.82
10	R\$ 1,236,852.99	R\$ nan	10.43%	R\$ 1,675,687.60	R\$ 438,834.61
11	R\$ 1,096,651.48	R\$ nan	9.25%	R\$ 1,485,742.69	R\$ 389,091.21
12	R\$ 548,955.00	R\$ nan	4.63%	R\$ 743,723.87	R\$ 194,768.87



3.9. Auditoria de Carteira: Identificação de Clientes Órfãos e Consultores Inativos

Este procedimento técnico realiza a validação da integridade da força de vendas para o ciclo atual. O objetivo é cruzar a base histórica de clientes de 2025 com o desempenho real de 2026, identificando lacunas de atendimento (clientes "órfãos") decorrentes de consultores que não registraram movimentação no ano vigente.

Destaques da Metodologia:

- **Mapeamento de Atividade:** O motor de processamento filtra o `df_vendas_lub_com_ano` para isolar os consultores que já realizaram faturamento em 2026, criando uma lista de profissionais ativos.
- **Deteção de Vacância:** O sistema marca no `df_master_campanha` todos os clientes cujo consultor responsável em 2025 não consta na lista de ativos de 2026, classificando-os como "Clientes Órfãos".
- **Mensuração de Risco Financeiro:** A auditoria consolida o `potencial_estrategico` desses clientes, permitindo que a gestão visualize o valor monetário que está sem cobertura comercial ativa (Potencial em Risco).
- **Sincronização e Reativação (De-Para):** O sistema identifica clientes que pertenciam ao Consultor A (2025), mas que registraram compras com o Consultor B em 2026. Nestes casos, o algoritmo realiza a reativação da carteira, transferindo automaticamente a meta de faturamento e o potencial estratégico para o consultor ativo.

In [151...

```
### **3.9. Auditoria de Carteira: Identificação de Órfãos via Atividade Global**

import pandas as pd

# 1. ENGENHARIA DE ATRIBUTO: Garantir o ano na base global saneada
df_vendas_global = df_vendas_clean.copy()
df_vendas_global['ano_faturamento'] = pd.to_datetime(df_vendas_global['data_emis
```

```

# 2. IDENTIFICAÇÃO DE CONSULTORES ATIVOS EM 2026 (Atividade em QUALQUER produto)
consultores_ativos_2026 = df_vendas_global[
    df_vendas_global['ano_faturamento'] == 2026
][['consultor']].unique()

# 3. MAPEAMENTO DA CARTEIRA REAL 2026 (Baseado na última compra de QUALQUER prod
carteira_atual_2026 = df_vendas_global[df_vendas_global['ano_faturamento'] == 20
    .sort_values('data_emissao')\
    .groupby('cnpj')\
    .agg({
        'consultor': 'last',
        'filial': 'last',
        'valor_liquido': 'sum'
    }).reset_index().rename(columns={
        'consultor': 'consultor_2026',
        'filial': 'filial_2026',
        'valor_liquido': 'faturado_total_2026'
    })

# 4. CRUZAMENTO COM O MASTER DE CAMPANHA
# Note: df_master_campanha contém a coluna 'pot_est' vinda do bloco 3.8.2
df_auditoria = pd.merge(
    df_master_campanha,
    carteira_atual_2026[['cnpj', 'consultor_2026', 'filial_2026', 'faturado_tota
on='cnpj',
how='left'
)

# 5. IDENTIFICAÇÃO DE MUDANÇA DE CARTEIRA (DE-PARA GLOBAL)
df_auditoria['mudou_carteira'] = (
    df_auditoria['consultor_2026'].notna() &
    (df_auditoria['consultor'] != df_auditoria['consultor_2026'])
)

# 6. ATUALIZAÇÃO DO MASTER (O novo dono assume o potencial de lubrificantes)
def atualizar_dono_global(row):
    if row['mudou_carteira']:
        return row['consultor_2026'], row['filial_2026']
    return row['consultor'], row['filial']

df_master_campanha[['consultor', 'filial']] = df_auditoria.apply(atualizar_dono_

# 7. CÁLCULO DE ÓRFÃOS REAIS
df_master_campanha['consultor_ativo_2026'] = df_master_campanha['consultor'].isi

clientes_orfaos = df_master_campanha[df_master_campanha['consultor_ativo_2026']
clientes_ativos_carteira = df_master_campanha[df_master_campanha['consultor_ativ

# 8. RELATÓRIO DE IMPACTO CONSOLIDADO (Sincronizado com 'pot_est')
print("-" * 60)
print(f"📊 AUDITORIA DE MOVIMENTAÇÃO: ATIVIDADE GLOBAL (TODOS OS PRODUTOS)")
print("-" * 60)
print(f"Total de Clientes Analisados: {len(df_master_campanha)}")
print(f"Clientes que MUDARAM de Consultor: {len(df_auditoria[df_auditoria['
# Correção: potencial_estrategico -> pot_est
print(f"Potencial de Lubrificantes Re-alocado: R$ {df_auditoria[df_auditoria['m
print("-" * 60)
print(f"Clientes com Consultor ATIVO (Global): {len(clientes_ativos_carteira):<
print(f"Clientes com Consultor INATIVO (Órfãos): {len(clientes_orfaos):<10} ⚠️ "

```

```
# Correção: potencial_estrategico -> pot_est
print(f"Potencial de Lub. em Risco (ÓRFÃO):      R$ {clientes_orfaos['pot_est'].s
print("-" * 60)

if len(df_auditoria[df_auditoria['mudou_carteira']]) > 0:
    print("\n📄 LOG DE MIGRAÇÃO (TOP 5):")
    # Correção: potencial_estrategico -> pot_est
    display(df_auditoria[df_auditoria['mudou_carteira']][['cnpj', 'cliente', 'co
```

🔍 AUDITORIA DE MOVIMENTAÇÃO: ATIVIDADE GLOBAL (TODOS OS PRODUTOS)

Total de Clientes Analisados:	976
Clientes que MUDARAM de Consultor:	295
Potencial de Lubrificantes Re-alocado:	R\$ 8,612,891.86

Clientes com Consultor ATIVO (Global): 876

Clientes com Consultor INATIVO (Órfãos): 100

Potencial de Lub. em Risco (ÓRFÃO): R\$ 407,259.16

📄 LOG DE MIGRAÇÃO (TOP 5):

	cnpj	cliente	consultor	consultor_2026	pot_est
4	00000236253760	CARLOS ROBERTO CORREA DAS NEVES	Carlos Rocha	Danillo Bermudes	5787.707222
8	00000506451615	OROSTRATO OLAVO SILVA BARBOSA	Hellen Campos	Nathalia Menezes	38624.904000
13	00000572705891	JOSE EDUARDO FERREIRA NETTO	Samara Souza	Tomaz Zei	35126.028000
14	00000778187764	ADALTO KIRMSE	Carlos Rocha	Danillo Bermudes	6048.459193
20	00001226454640	VANDERLEI LOPES GOMES	Hellen Campos	Yan Marques	2036.256000

3.9.1. Análise de Vulnerabilidade: Segmentação por Status de Gestão

Após a sincronização da carteira, este módulo realiza um "raio-x" da saúde comercial da base. A inteligência aqui aplicada permite separar os clientes que estão performando daqueles que representam um risco real de perda de market share.

Critérios de Classificação:

- **Ativo (Faturado em 2026):** Clientes que já registraram compras no ano corrente. A meta aqui foca em Crescimento e manutenção do share.
- **Aguardando Visita:** O consultor responsável está ativo na empresa (com vendas em outros CNPJs), mas o cliente específico de 2025 ainda não comprou em 2026. Representa uma falha de execução ou prospecção de um consultor que está "no trecho".

- **ÓRFÃO REAL:** O nível crítico de vulnerabilidade. O cliente não comprou em 2026 e o consultor responsável em 2025 está inativo (sem vendas em 2026). Este faturamento está "abandonado" e requer re-alocação imediata para evitar o Churn definitivo.

Objetivos do Relatório:

- **Quantificar o GAP de Esforço:** Diferença entre a meta projetada e o histórico, evidenciando o desafio comercial de cada segmento.
- **Isolar o Risco Financeiro:** Identificar o valor monetário exato das metas que estão sem dono (Órfãos Reais).
- **Priorização de Ataque:** Listar os principais CNPJs órfãos por ordem de potencial, permitindo uma redistribuição estratégica baseada em volume financeiro.

```
In [152... ### **3.9.1. Análise de Vulnerabilidade e Clientes Órfãos**

# 1. MAPEAMENTO DE ATIVIDADE E STATUS
consultores_ativos_2026 = df_vendas_clean[
    pd.to_datetime(df_vendas_clean['data_emissao']).dt.year == 2026
][['consultor']].unique()

def classificar_status_vulnerabilidade(row):
    comprou_2026 = row['realizado_2026_acumulado'] > 0
    consultor_no_trecho = row['consultor'] in consultores_ativos_2026
    if comprou_2026: return "Ativo (Faturado em 2026)"
    elif consultor_no_trecho: return "Aguardando Visita (Consultor Ativo / S/ Co
    else: return "ÓRFÃO REAL (Consultor Inativo em 2026)"

df_master_campanha['status_gestao'] = df_master_campanha.apply(classificar_statu

# 2. RELATÓRIO DE RISCO
resumo_risco = df_master_campanha.groupby('status_gestao').agg({
    'cnpj': 'count', 'faturamento_2025_real': 'sum', 'pot_est': 'sum'
}).rename(columns={'cnpj': 'Qtd Clientes', 'pot_est': 'Potencial Estratégico'})

print("-" * 75)
print(f"⚠️ DIAGNÓSTICO DE VULNERABILIDADE: RISCO SOBRE O POTENCIAL ESTRATÉGICO")
print("-" * 75)

# Formatação Seletiva para evitar 'R$' em Qtd Clientes
display(resumo_risco.style.format({
    'Qtd Clientes': '{:,}', # Quantitativo
    'faturamento_2025_real': 'R$ {:,2f}',
    'Potencial Estratégico': 'R$ {:,2f}'
}).apply(lambda x: ['background-color: #FFDE00; color: #333333; font-weight: bol
    if x.name == 'ÓRFÃO REAL (Consultor Inativo em 2026)'
    else '' for i in x], axis=1))

# 3. TOP 5 CLIENTES ÓRFÃOS
df_detalhe_orfaos = df_master_campanha[df_master_campanha['status_gestao'] == "Ó
top_5_orfaos = df_detalhe_orfaos.sort_values(by='pot_est', ascending=False).head

print("\n--- 🚨 TOP 5 PRIORIDADES DE RE-ALOCAÇÃO (MAIOR POTENCIAL EM RISCO) ---")
display(top_5_orfaos[['filial', 'consultor', 'cnpj', 'cliente', 'faturamento_202
    'faturamento_2025_real': 'R$ {:,2f}'],
```

```
'pot_est': 'R$ {:,.2f}'
)))
```

⚠️ DIAGNÓSTICO DE VULNERABILIDADE: RISCO SOBRE O POTENCIAL ESTRATÉGICO

			Qtd Clientes	faturamento_2025_real	Potencial Estratégico	
status_gestao						
Aguardando Visita (Consultor Ativo / S/ Compra)			636	R\$ 4,626,227.47	R\$ 6,370,874.74	
Ativo (Faturado em 2026)			240	R\$ 6,939,112.22	R\$ 9,284,723.22	
ÓRFÃO REAL (Consultor Inativo em 2026)			100	R\$ 290,923.27	R\$ 407,259.16	
--- 🚨 TOP 5 PRIORIDADES DE RE-ALOCAÇÃO (MAIOR POTENCIAL EM RISCO) ---						
	filial	consultor	cnpj	cliente	faturamento_2025_real	
361	0203 - Serra	Eliane Gils	00679427000834	AMBIPAR ENVIRONMENTAL SOLUTIONS SOLUCO	R\$ 26,513.71	31,
704	0201 - Contagem	Breno Costa	23373000000132	VAMOS LOCACAO DE CAMINHOS MAQUINAS E EQUIPAMENTOS S.A.	R\$ 1,585.47	30,
801	0202 - Tanguá	Diego Vicente	33081704003020	SIKA S A	R\$ 10,632.99	12,
480	0203 - Serra	Carlos Rocha	08294854000130	WB LOCACAO DE MAQUINAS LTDA	R\$ 8,665.85	10,
18	0204 - Urberlândia	Lilian Rosa	00001003399649	PAULO VELOSO DOS SANTOS	R\$ 7,585.58	9,

In [153...

```
### **3.9.2. Análise de Vulnerabilidade e Clientes Órfãos**

# 1. Filtrar apenas os Órfãos Reais
df_orfaos_detalhado = df_master_campanha[
    df_master_campanha['status_gestao'] == "ÓRFÃO REAL (Consultor Inativo em 202
].copy()

# 2. Relatório Consolidado: Sincronizando nomes de colunas do bloco 3.8.2
resumo_tipo_gap = df_orfaos_detalhado.groupby('classificacao_gap').agg({
    'cnpj': 'count',
    'faturamento_2025_real': 'sum',
    'pot_est': 'sum',      # Sincronizado com 3.8.2
    'gap_anual': 'sum'     # Sincronizado com 3.8.2
}).rename(columns={
    'cnpj': 'Qtd Clientes',
    'faturamento_2025_real': 'Histórico 2025',
```

```

    'pot_est': 'Meta 2026',
    'gap_anual': 'GAP de Ataque'
})

print("--- 📊 RESUMO DOS ÓRFÃOS POR NATUREZA DE GAP ---")

# 3. Exibição com formatação seletiva (Quantitativo vs Monetário)
display(resumo_tipo_gap.style.format({
    'Qtd Clientes': '{:,}',
    'Histórico 2025': 'R$ {:,}.2f}',
    'Meta 2026': 'R$ {:,}.2f}',
    'GAP de Ataque': 'R$ {:,}.2f}'
}).background_gradient(cmap='YlGn', subset=['GAP de Ataque'])))

```

--- 📊 RESUMO DOS ÓRFÃOS POR NATUREZA DE GAP ---

	Qtd Clientes	Histórico 2025	Meta 2026	GAP de Ataque
classificacao_gap				
Crescimento (Faturado > Potencial)	15	R\$ 69,371.12	R\$ 83,245.34	R\$ 13,874.22
Prospecção (Sem Dados de Frota)	39	R\$ 113,235.42	R\$ 135,882.50	R\$ 22,647.08
Recuperação (Faturado < Potencial)	46	R\$ 108,316.73	R\$ 188,131.31	R\$ 79,814.58

In [154...

```

### 3.9.3

import pandas as pd
import numpy as np

# 1. CRIANDO A BASE DE COMPARAÇÃO GLOBAL (vendas_empresa_ano)
# Consolidamos o faturamento de todos os produtos para identificar se o cliente
df_vendas_global = df_vendas_clean.copy() # Base global de todas as notas fiscais
df_vendas_global['ano'] = pd.to_datetime(df_vendas_global['data_emissao']).dt.year

vendas_empresa_ano = df_vendas_global.groupby(['cnpj', 'ano'])['valor_liquido'].
vendas_empresa_ano = vendas_empresa_ano.rename(columns={2025: 'Venda_Total_Inova

# 2. CRUZAMENTO COM A TABELA MESTRE (Sincronizado com pot_est)
df_analise_viva = pd.merge(
    df_master_campanha[[
        'cnpj', 'cliente', 'consultor', 'filial',
        'faturamento_2025_real', 'realizado_2026_acumulado',
        'pot_est', 'status_gestao'
    ]],
    vendas_empresa_ano[['cnpj', 'Venda_Total_Inova_2025', 'Venda_Total_Inova_202
on='cnpj',
how='left'
).fillna(0)

# 3. LÓGICA DE CLASSIFICAÇÃO DE FIDELIDADE

def auditoria_movimentacao(row):
    # Caso 1: Cliente não comprou nada na Inova em 2026
    if row['Venda_Total_Inova_2025'] > 0 and row['Venda_Total_Inova_2026'] == 0:
        return "CHURN GLOBAL (Abandono Total)"

```

```
# Caso 2: Comprou outros produtos em 2026, mas ZERO Lubrificantes
elif row['realizado_2026_acumulado'] == 0 and row['Venda_Total_Inova_2026']:
    return "CHURN LUB (Perda de Share p/ Concorrência)"

# Caso 3: Já comprou Lubrificantes em 2026
elif row['realizado_2026_acumulado'] > 0:
    return "RETIDO (Ativo em Lubrificantes)"

return "OUTROS / NOVOS"

df_analise_viva['perfil_fidelidade'] = df_analise_viva.apply(auditoria_movimenta

# 4. RESUMO EXECUTIVO DO CHURN
resumo_churn = df_analise_viva.groupby('perfil_fidelidade').agg({
    'cnpj': 'count',
    'pot_est': 'sum'
}).rename(columns={'cnpj': 'Qtd Clientes', 'pot_est': 'Potencial em Risco (Ano)'

# 5. EXIBIÇÃO FORMATADA
print("-" * 60)
print(f"📊 DIAGNÓSTICO DE FIDELIDADE E SAÚDE DA CARTEIRA 2026")
print("-" * 60)

display(resumo_churn.style.format({
    'Qtd Clientes': '{:,}',
    'Potencial em Risco (Ano)': 'R$ {:,.2f}'
}).background_gradient(cmap='gray', subset=['Potencial em Risco (Ano)'])))
```

📊 DIAGNÓSTICO DE FIDELIDADE E SAÚDE DA CARTEIRA 2026

	Qtd Clientes	Potencial em Risco (Ano)
perfil_fidelidade		
CHURN GLOBAL (Abandono Total)	503	R\$ 2,893,799.25
CHURN LUB (Perda de Share p/ Concorrência)	233	R\$ 3,884,334.65
RETIDO (Ativo em Lubrificantes)	240	R\$ 9,284,723.22

3.10. Consolidação da Matriz de Metas: Inteligência Sazonal e Governança Ativa

Esta etapa final do motor de ETL transforma o **Potencial Estratégico** individual em um cronograma tático de faturamento mensal para o ciclo comercial de 2026. O modelo utiliza uma **distribuição ponderada por cliente**, baseada na curva real de consumo de 2025, garantindo que as metas sejam aderentes aos ciclos de manutenção, safra e sazonalidade da frota.

Para garantir a governança e a aplicabilidade prática, esta etapa foi dividida em dois módulos operacionais:

- 1. **3.10.1. Matriz de Visão Gerencial:** Um painel consolidado e hierarquizado por Filial e Consultor, ideal para acompanhamento de liderança.

2. **3.10.2. Exportação Granular Mensalizada:** A geração da base de dados final para o usuário ("Tidy Data"), com valores fracionados mês a mês, pronta para ingestão em ferramentas de BI (PowerBI, Tabelas Dinâmicas) sem o risco de duplicidade matemática.



Destaques da Metodologia e Governança:

- **Algoritmo de Distribuição (Regra 12 Meses):** Diferente de uma média linear, o sistema aplica o peso histórico específico de cada mês para cada CNPJ, preservando a integridade do potencial anual no somatório final.
- **Filtro de Soberania Ativa:** A matriz exclui automaticamente consultores inativos (Órfãos), garantindo que a meta financeira da companhia seja atribuída apenas à força de vendas com capacidade de execução em 2026.
- **Hierarquia de Dados:** Os resultados são clusterizados por **Filial**, permitindo que os gerentes de unidade identifiquem desvios de meta de forma granular.
- **Auditoria de Qualidade (Data Quality):** O modelo conta com uma trava de segurança rigorosa que confronta o Potencial Total contra o Saldo de Ativos e Órfãos, assegurando **erro zero (R\$ 0,00 de diferença)** no processamento financeiro.

3.10.1. Geração da Matriz Gerencial de Metas

Neste bloco, o algoritmo projeta as metas mensais aplicando o peso de sazonalidade de 2025 de cada cliente. Em seguida, os dados são pivotados (agrupados) em uma visão macro por **Filial** e **Consultor**. O resultado é um painel de visualização rápida formatado com a identidade visual executiva (Padrão John Deere), permitindo a validação visual do volume de metas distribuído ao longo do ano.

```
In [ ]: ### 3.10.1. Geração da Matriz Gerencial de Metas

import pandas as pd
import numpy as np

# --- 0. DEFINIÇÃO DA FUNÇÃO DE ESTILO (Padrão John Deere Executivo) ---
def style_john_deere(styler):
    jd_green = '#367C2B'
    jd_yellow = '#FFDE00'
    zebra_light = '#e6e6e6' # Efeito zebra sutil
    header_index_bg = '#333333' # Cinza para índices

    # Configuração de Base
    styler.set_properties(**{'background-color': '#ffffff', 'color': '#333333'},
    styler.format(lambda val: f'R$ {val:,.2f}'.replace(',', 'X').replace('.', ','),

    # Estilização dos Cabeçalhos e Índices (Filial e Consultor)
    styler.set_table_styles([
        {'selector': 'th.row_heading', 'props': [('background-color', header_ind
        {'selector': 'th.col_heading', 'props': [('background-color', '#e6e6e6')
    ]})

    # Lógica de Linhas (Zebra + Totais Institucionais)
    def apply_row_styles(row):
```



```

        idx = row.name
        row_pos = df_final.index.get_loc(idx)

        # Total Global (Amarelo JD)
        if "TOTAL INOVA" in str(idx[1]):
            return [f'background-color: {jd_yellow}; color: {jd_green}; font-wei

        # Total Filial (Verde JD)
        if "TOTAL FILIAL" in str(idx[1]):
            return [f'background-color: {jd_green}; color: white; font-weight: b

        # Zebra Sutil
        if row_pos % 2 == 0:
            return [f'background-color: {zebra_light}'] * len(row)

        return [''] * len(row)

    styler.apply(apply_row_styles, axis=1)

    # Gradiente na coluna Total para indicar volume de metas
    styler.background_gradient(cmap='Greens', subset=pd.IndexSlice[df_final.inde

    return styler

# --- 1. CÁLCULO DA MATRIZ DE PESOS ---
df_2025_dist = df_vendas_lub_com_ano[df_vendas_lub_com_ano['ano_faturamento'] ==
df_2025_dist['mes'] = df_2025_dist['data_emissao'].dt.month

venda_mensal_matriz = df_2025_dist.groupby(['cnpj', 'mes'])['valor_liquido'].sum
venda_anual_total_cnpj = venda_mensal_matriz.sum(axis=1)
pesos_mensais_full = venda_mensal_matriz.div(venda_anual_total_cnpj, axis=0)

# --- 2. CRIAÇÃO DO CRONOGRAMA ---
df_expansao = df_master_campanha.copy()
meses_colunas = [f'01/{m:02d}/2026' for m in range(1, 13)]

for m in range(1, 13):
    col_nome = f'01/{m:02d}/2026'
    if m in pesos_mensais_full.columns:
        pesos_do_mes = df_expansao['cnpj'].map(pesos_mensais_full[m]).fillna(1/1
    else:
        pesos_do_mes = pd.Series(1/12, index=df_expansao.index)

    # Aplicação do Potencial Estratégico (pot_est)
    df_expansao[col_nome] = pesos_do_mes * df_expansao['pot_est']

# --- 3. FILTRAGEM E PIVOTAGEM ---
df_expansao_ativa = df_expansao[df_expansao['consultor'].isin(consultores_ativos

df_expansao_tidy = df_expansao_ativa.melt(
    id_vars=['filial', 'consultor', 'cnpj', 'cliente'],
    value_vars=meses_colunas,
    var_name='Mês_Ref',
    value_name='Valor_Meta'
)

matriz_base = df_expansao_tidy.pivot_table(
    index=['filial', 'consultor'],
    columns='Mês_Ref',
    values='Valor_Meta',

```

```

        aggfunc='sum'
    ).fillna(0)

# --- 4. CONSOLIDAÇÃO DE TOTAIS ---
totais_por_filial = matriz_base.groupby(level=0).sum()
totais_por_filial.index = pd.MultiIndex.from_tuples([(f, 'zz_TOTAL FILIAL') for
df_final = pd.concat([matriz_base, totais_por_filial]).sort_index()
df_final['Total'] = df_final.sum(axis=1)

total_global_row = totais_por_filial.sum()
total_global_row['Total'] = total_global_row.sum()
df_final.loc[('zz_GLOBAL', 'TOTAL INOVA'), :] = total_global_row

# --- 5. VALIDAÇÃO E EXIBIÇÃO ---
# Verificação de Integridade
potencial_total_inicial = df_master_campanha['pot_est'].sum()
valor_orfaos = df_master_campanha[~df_master_campanha['consultor']].isin(consulto
valor_esperado_ativos = potencial_total_inicial - valor_orfaos
valor_real_na_matriz = df_final.loc[('zz_GLOBAL', 'TOTAL INOVA'), 'Total']

print(f"--- 🚫 VALIDAÇÃO MATEMÁTICA DA MATRIZ ---")
print(f"1. Potencial Estratégico Total:    R$ {potencial_total_inicial:,.2f}")
print(f"2. (-) Valor dos Clientes Órfãos:    R$ {valor_orfaos:,.2f}")
print(f"3. (=) SALDO ESPERADO (Ativos):      R$ {valor_esperado_ativos:,.2f}")
print(f"4. VALOR REAL NA MATRIZ:              R$ {valor_real_na_matriz:,.2f}")
print(f"--- Diferença Final:                  R$ {abs(valor_esperado_ativos - valor_

# Ajuste Final de Índices para Display
novos_indices = [(f.replace('zz_', ''), c.replace('zz_', '')) for f, c in df_fin
df_final.index = pd.MultiIndex.from_tuples(novos_indices, names=['Filial', 'Cons

display(style_john_deere(df_final[meses_colunas + ['Total']].style))

--- 🚫 VALIDAÇÃO MATEMÁTICA DA MATRIZ ---
1. Potencial Estratégico Total:    R$ 16,062,857.12
2. (-) Valor dos Clientes Órfãos:  R$ 407,259.16
3. (=) SALDO ESPERADO (Ativos):    R$ 15,655,597.96
4. VALOR REAL NA MATRIZ:           R$ 15,655,597.96
--- Diferença Final:               R$ 0.00

```

		Mês_Ref	01/01/2026	01/02/2026	01/03/2026	01/04/2026	01/05/2026
Filial	Consultor						
0201 - Contagem	Ana Souza	R\$ 0,00	R\$ 2.499,98	R\$ 1.242,62	R\$ 0,00	R\$ 6.426,00	
	Andre Bessas	R\$ 0,00	R\$ 0,00	R\$ 1.144,14	R\$ 0,00	R\$ 0,00	
	Aurelio Costa	R\$ 12.016,44	R\$ 108.472,85	R\$ 28.815,83	R\$ 96.631,69	R\$ 36.006,00	
	Bianca Cruz	R\$ 0,00	R\$ 4.244,60	R\$ 0,00	R\$ 0,00	R\$ 2.028,00	
	Danillo Bermudes	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0,00	
	Graziane Leite	R\$ 49.621,30	R\$ 77.737,48	R\$ 51.278,08	R\$ 62.443,64	R\$ 147.888,00	
	Gustavo Hamdan	R\$ 5.371,42	R\$ 19.899,02	R\$ 14.107,36	R\$ 44.585,54	R\$ 20.350,00	
	Hellen Campos	R\$ 21.683,00	R\$ 22.744,49	R\$ 32.426,86	R\$ 21.132,60	R\$ 30.590,00	
	Kelly Santos	R\$ 36.156,17	R\$ 16.221,66	R\$ 15.379,69	R\$ 6.189,11	R\$ 8.586,00	
	Luana Costa	R\$ 2.190,96	R\$ 0,00	R\$ 3.820,75	R\$ 5.627,53	R\$ 942,00	
	Luana Machado	R\$ 0,00	R\$ 7.073,06	R\$ 1.776,17	R\$ 2.252,05	R\$ 5.911,00	
	Luis Barbosa	R\$ 82.735,96	R\$ 66.791,08	R\$ 70.293,25	R\$ 63.639,05	R\$ 78.507,00	
	Luis Campos	R\$ 667,18	R\$ 2.259,56	R\$ 1.292,22	R\$ 9.765,06	R\$ 14.451,00	
	Marcos Oliveira	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 2.512,81	R\$ 2.059,00	
	Moises Carvalho	R\$ 134.037,30	R\$ 147.246,67	R\$ 108.386,92	R\$ 101.473,03	R\$ 123.964,00	
	Paula Reis	R\$ 84.998,81	R\$ 119.860,50	R\$ 67.211,93	R\$ 118.363,74	R\$ 91.997,00	
	Pedro Candido	R\$ 37.863,62	R\$ 7.595,83	R\$ 15.127,64	R\$ 7.410,38	R\$ 2.122,00	
	Pedro Lima	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0,00	
	Samara Souza	R\$ 2.321,26	R\$ 0,00	R\$ 888,08	R\$ 5.960,44	R\$ 8.136,00	
	Yan Marques	R\$ 10.584,96	R\$ 21.013,18	R\$ 12.288,23	R\$ 31.647,24	R\$ 3.849,00	

		Mês_Ref	01/01/2026	01/02/2026	01/03/2026	01/04/2026	01/05/2026
Filial	Consultor						
0202 - Tanguá	TOTAL FILIAL	R\$ 480.248,38	R\$ 623.659,97	R\$ 425.479,77	R\$ 579.633,89	R\$ 583.820,00	
	Andre Alves	R\$ 22.459,04	R\$ 41.483,88	R\$ 15.642,64	R\$ 15.851,90	R\$ 8.263,00	
	Douglas Rodrigues	R\$ 964,22	R\$ 922,94	R\$ 19.695,42	R\$ 33.285,19	R\$ 15.185,00	
	Kelly Santos	R\$ 0,00	R\$ 12.220,57	R\$ 14.736,62	R\$ 0,00	R\$ 0,00	
	Marcelo Fiuzas	R\$ 22.691,35	R\$ 13.698,59	R\$ 9.401,21	R\$ 9.775,72	R\$ 6.690,00	
	Yan Marques	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0,00	
	TOTAL FILIAL	R\$ 46.114,61	R\$ 68.325,99	R\$ 59.475,90	R\$ 58.912,80	R\$ 30.140,00	
	Amanda Lourenco	R\$ 1.897,13	R\$ 702,30	R\$ 702,30	R\$ 0,00	R\$ 2.635,00	
	Ana Souza	R\$ 1.014,22	R\$ 0,00	R\$ 1.014,22	R\$ 0,00	R\$ 0,00	
	Danillo Bermudes	R\$ 67.479,96	R\$ 17.207,06	R\$ 132.210,78	R\$ 64.089,05	R\$ 56.392,00	
0203 - Serra	Gabriel Nascimento	R\$ 2.469,88	R\$ 4.875,42	R\$ 1.912,75	R\$ 0,00	R\$ 0,00	
	Gustavo Hamdan	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0,00	
	Hellen Campos	R\$ 3.904,61	R\$ 0,00	R\$ 0,00	R\$ 3.678,10	R\$ 0,00	
	Luciana Silva	R\$ 7.597,79	R\$ 1.017,14	R\$ 4.674,01	R\$ 775,67	R\$ 9.707,00	
	Paula Reis	R\$ 15.470,24	R\$ 5.995,72	R\$ 10.876,24	R\$ 7.843,52	R\$ 5.468,00	
	Vinicius Lenzi	R\$ 18.163,10	R\$ 30.431,99	R\$ 37.628,28	R\$ 16.205,45	R\$ 29.725,00	
	Yan Marques	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0,00	
	TOTAL FILIAL	R\$ 117.996,92	R\$ 60.229,63	R\$ 189.018,57	R\$ 92.591,78	R\$ 103.929,00	
	Bruna Fagundes	R\$ 10.898,32	R\$ 23.426,46	R\$ 21.420,48	R\$ 30.237,35	R\$ 131.173,00	
	Danilo Neto	R\$ 21.219,86	R\$ 14.360,36	R\$ 8.972,44	R\$ 23.547,07	R\$ 2.795,00	
0204 - Urberlândia							

		Mês_Ref	01/01/2026	01/02/2026	01/03/2026	01/04/2026	01/05/2026
Filial	Consultor						
	Graziane Leite	R\$ 13.255,31	R\$ 5.533,57	R\$ 0,00	R\$ 15.309,17	R\$ 440	
	Hellen Campos	R\$ 6.235,97	R\$ 7.297,80	R\$ 2.983,37	R\$ 1.462,04	R\$ 1.922	
	Luis Barbosa	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0	
	Luis Campos	R\$ 6.559,37	R\$ 5.608,60	R\$ 11.836,38	R\$ 2.496,65	R\$ 2.754	
	Marcus Borges	R\$ 12.883,61	R\$ 5.544,03	R\$ 8.266,44	R\$ 4.794,71	R\$ 14.035	
	Moises Carvalho	R\$ 0,00	R\$ 23.958,00	R\$ 0,00	R\$ 0,00	R\$ 0	
	Samara Souza	R\$ 3.419,53	R\$ 4.138,20	R\$ 2.209,15	R\$ 4.179,24	R\$ 0	
	Thiago Gizzi	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0	
	Tomaz Zei	R\$ 89.256,88	R\$ 121.303,39	R\$ 60.521,62	R\$ 161.337,29	239.929	
	Wenderson Silva	R\$ 23.493,51	R\$ 60.363,45	R\$ 51.350,54	R\$ 43.359,76	111.007	
	TOTAL FILIAL	R\$ 187.222,35	R\$ 271.533,86	R\$ 167.560,42	R\$ 286.723,27	504.059	
0210 - Pouso Alegre	Ana Souza	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0	
	Hellen Campos	R\$ 2.378,72	R\$ 5.385,70	R\$ 2.796,64	R\$ 10.722,58	R\$ 0	
	Kelly Santos	R\$ 1.306,27	R\$ 0,00	R\$ 639,10	R\$ 0,00	R\$ 1.966	
	Luana Costa	R\$ 4.546,16	R\$ 2.723,32	R\$ 14.989,86	R\$ 18.659,46	R\$ 2.839	
	Luis Campos	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 702,30	R\$ 6.617	
	Nathalia Menezes	R\$ 5.788,51	R\$ 20.430,67	R\$ 24.586,83	R\$ 29.349,75	R\$ 20.903	
	TOTAL FILIAL	R\$ 14.019,67	R\$ 28.539,68	R\$ 43.012,42	R\$ 59.434,09	32.327	
0211 - CRC	Andre Bessas	R\$ 6.783,48	R\$ 0,00	R\$ 2.111,56	R\$ 1.302,76	R\$ 8.425	
	Lucas Faria	R\$ 35.075,39	R\$ 55.266,90	R\$ 39.995,98	R\$ 59.041,76	R\$ 25.235	

		Mês_Ref	01/01/2026	01/02/2026	01/03/2026	01/04/2026	01/05/2026
Filial	Consultor						
	Luciana Silva		R\$ 0,00	R\$ 0,00	R\$ 11.490,60	R\$ 1.379,60	R\$ 2.228,00
	Marcelo Fiuzas		R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0,00	R\$ 0,00
	TOTAL FILIAL		R\$ 41.858,87	R\$ 55.266,90	R\$ 53.598,13	R\$ 61.724,12	R\$ 35.889,00
0212 - CSN	Samara Souza		R\$ 43.209,86	R\$ 48.878,71	R\$ 0,00	R\$ 7.208,98	R\$ 35.324,00
	TOTAL FILIAL		R\$ 43.209,86	R\$ 48.878,71	R\$ 0,00	R\$ 7.208,98	R\$ 35.324,00
0301 - Contagem	Luiz Junior		R\$ 73.442,67	R\$ 29.786,61	R\$ 0,00	R\$ 0,00	R\$ 0,00
	TOTAL FILIAL		R\$ 73.442,67	R\$ 29.786,61	R\$ 0,00	R\$ 0,00	R\$ 0,00
GLOBAL	TOTAL INOVA		R\$ 1.004.113,33	R\$ 1.186.221,36	R\$ 938.145,21	R\$ 1.146.228,95	R\$ 1.325.491,00

3.10.2. Exportação Mensalizada Inteligente e Validação Matemática

Para que os dados sejam perfeitamente lidos por Tabelas Dinâmicas (Excel) e ferramentas de BI (PowerBI), a base não pode conter métricas *anuais* repetidas em linhas *mensais* (o que causaria a multiplicação irreal do faturamento).

Este bloco aplica uma **Engenharia de Fração Sazonal**:

1. Converte a matriz de 12 colunas em 12 linhas por cliente (`melt`).
2. Descobre o peso exato de sazonalidade de cada mês (`Meta_Mensal / Potencial_Anual`).
3. Multiplica o Faturamento de 2025, o Potencial Estratégico e a Oportunidade (GAP) por esse peso, garantindo que a soma dos 12 meses reflita exatamente o valor do ano.
4. Finaliza com uma rigorosa **Trava de Qualidade de Dados (Data Quality)**, subtraindo os clientes Órfãos do Potencial Total da empresa e garantindo que o arquivo exportado possua exatos **R\$ 0,00 de diferença** com a matemática global.

In [162...

```
import pandas as pd
import numpy as np

# =====
# PASSO 1: RECONSTRUIR AS COLUNAS DOS MESES NA BASE MAIS ATUALIZADA
# =====
# Pegamos a base mestra que já tem o 'status_gestao' e 'gap_anual' (Etapa 3.9)
df_expansao = df_master_campanha.copy()
```

```

# Recriamos as 12 colunas de meses usando a matriz de pesos 'pesos_mensais_full'
meses_colunas = []
for m in range(1, 13):
    col_nome = f'01/{m:02d}/2026'
    meses_colunas.append(col_nome)

# Pega o peso do cliente para o mês, se não tiver histórico, recebe peso lin
if m in pesos_mensais_full.columns:
    pesos_do_mes = df_expansao['cnpj'].map(pesos_mensais_full[m]).fillna(1/12)
else:
    pesos_do_mes = pd.Series(1/12, index=df_expansao.index)

# A Meta do Mês é o Potencial Estratégico Anual * Peso do Mês
df_expansao[col_nome] = pesos_do_mes * df_expansao['pot_est']

# =====
# PASSO 2: FILTRAR ATIVOS E TRANSFORMAR (Wide para Long)
# =====
# Filtramos apenas os consultores ativos em 2026
consultores_ativos_2026 = df_vendas_clean[pd.to_datetime(df_vendas_clean['data_e
df_expansao_ativa = df_expansao[df_expansao['consultor'].isin(consultores_ativos

# Realizamos o 'melt' incluindo Faturamento e GAP nas variáveis de identificação
df_export_pivot = df_expansao_ativa.melt(
    id_vars=[
        'filial', 'consultor', 'cnpj', 'cliente',
        'status_gestao', 'classificacao_gap',
        'faturamento_2025_real', 'pot_est', 'gap_anual'
    ],
    value_vars=meses_colunas,
    var_name='Mes_Referencia',
    value_name='Meta_Mensal'
)

# =====
# PASSO 3: MENSALIZAÇÃO DOS VALORES (Inteligência Sazonal Exata)
# =====
df_export_pivot['Data_Referencia'] = pd.to_datetime(df_export_pivot['Mes_Referen
df_export_pivot['Mes_Numero'] = df_export_pivot['Data_Referencia'].dt.month
df_export_pivot['Trimestre'] = df_export_pivot['Data_Referencia'].dt.quarter

# Descobrimos o peso fracionado de cada Linha (Mês)
df_export_pivot['Peso_Sazonal'] = np.where(
    df_export_pivot['pot_est'] > 0,
    df_export_pivot['Meta_Mensal'] / df_export_pivot['pot_est'],
    1/12
)

# Fracionamos o Faturamento e o Gap pelo peso exato de cada mês
df_export_pivot['Faturado_Mes_2025'] = df_export_pivot['faturamento_2025_real']
df_export_pivot['Oportunidade_GAP_Mensal'] = df_export_pivot['gap_anual'] * df_e

# =====
# PASSO 4: LIMPEZA E PADRONIZAÇÃO DE NOMES
# =====
df_export_pivot['filial'] = df_export_pivot['filial'].astype(str).str.replace('z
df_export_pivot['consultor'] = df_export_pivot['consultor'].astype(str).str.repl

renomeio_colunas = {

```

```

        'filial': 'Filial',
        'consultor': 'Consultor',
        'cnpj': 'CNPJ',
        'cliente': 'Cliente',
        'status_gestao': 'Status_Gestao',
        'classificacao_gap': 'Classificacao_Estrategica'
    }
df_export_pivot = df_export_pivot.rename(columns=renomeio_colunas)

# Colunas restritas apenas aos valores mensais fracionados (pronto para Tabela D
colunas_excel = [
    'Filial', 'Consultor', 'CNPJ', 'Cliente', 'Status_Gestao', 'Classificacao_Es
    'MesReferencia', 'Mes_Numero', 'Trimestre',
    'Faturado_Mes_2025', 'Oportunidade_GAP_Mensal', 'Meta_Mensal'
]

df_final_excel = df_export_pivot[colunas_excel].copy()

# =====
# PASSO 5: VALIDAÇÃO MATEMÁTICA E TRAVA DE SEGURANÇA (Data Quality)
# =====
potencial_total_inicial = df_master_campanha['pot_est'].sum()
valor_orfaos = df_master_campanha[~df_master_campanha['consultor'].isin(consulto

valor_esperado_ativos = potencial_total_inicial - valor_orfaos
valor_real_na_matriz = df_final_excel['Meta_Mensal'].sum()

diferenca_final = abs(valor_esperado_ativos - valor_real_na_matriz)

# Print do Relatório Executivo
print("-" * 60)
print("🔵 VALIDAÇÃO MATEMÁTICA DA MATRIZ")
print("-" * 60)
print(f"1. Potencial Estratégico Total:    R$ {potencial_total_inicial:,.2f}")
print(f"2. (-) Valor dos Clientes Órfãos: R$ {valor_orfaos:,.2f}")
print(f"3. (=) SALDO ESPERADO (Ativos):    R$ {valor_esperado_ativos:,.2f}")
print(f"4. VALOR REAL NA MATRIZ:           R$ {valor_real_na_matriz:,.2f}")
print("-" * 60)
print(f"Diferença Final:                  R$ {diferenca_final:,.2f}\n")

# A TRAVA DE SEGURANÇA ACONTECE AQUI:
# Usamos 0.05 (5 centavos) como limite para ignorar pequenos erros de dízima do
if diferenca_final > 0.05:
    raise ValueError(f"❌ ERRO DE INTEGRIDADE: A diferença matemática é de R$ {

# =====
# PASSO 6: EXPORTAÇÃO PARA XLSX (Apenas se passar na trava acima)
# =====
nome_arquivo = "Base_Metas_Inova_2026_Mensalizada.xlsx"
df_final_excel.to_excel(nome_arquivo, index=False)

print("✅ EXPORTAÇÃO MENSALIZADA CONCLUÍDA COM SUCESSO")
print(f"Arquivo gerado: {nome_arquivo}")
print(f"Total de Linhas no Excel: {len(df_final_excel):,}")

# Exibição no notebook para visualização rápida
display(df_final_excel.head(5).style.format({
    'Faturado_Mes_2025': 'R$ {:, .2f}',
    'Oportunidade_GAP_Mensal': 'R$ {:, .2f}',

```



```
'Meta_Mensal': 'R$ {:,.2f}'  
)))
```

VALIDAÇÃO MATEMÁTICA DA MATRIZ

- 1. Potencial Estratégico Total: R\$ 16,062,857.12
- 2. (-) Valor dos Clientes Órfãos: R\$ 407,259.16
- 3. (=) SALDO ESPERADO (Ativos): R\$ 15,655,597.96
- 4. VALOR REAL NA MATRIZ: R\$ 15,655,597.96

Diferença Final: R\$ 0.00

EXPORTAÇÃO MENSALIZADA CONCLUÍDA COM SUCESSO
Arquivo gerado: Base_Metas_Inova_2026_Mensalizada.xlsx
Total de Linhas no Excel: 10,512

	Filial	Consultor	CNPJ	Cliente	Status_Gestao	Classificacao_Estrate
0	0201 - Contagem	Moises Carvalho	00000143983628	MARCOS FLAVIO SCHMITZ	Aguardando Visita (Consultor Ativo / S/ Compra)	Prospecção (Sem D de F
1	0203 - Serra	Ana Souza	00000169904717	ELMO TADEU GRECCO	Aguardando Visita (Consultor Ativo / S/ Compra)	Crescimento (Fatura Potenci
2	0204 - Urberlândia	Tomaz Zei	00000182280624	LINDOMAR DE MORAES	Aguardando Visita (Consultor Ativo / S/ Compra)	Prospecção (Sem D de F
3	0204 - Urberlândia	Samara Souza	00000221674802	MAMEDE MOREIRA BORGES DA COSTA	Aguardando Visita (Consultor Ativo / S/ Compra)	Prospecção (Sem D de F
4	0203 - Serra	Danillo Bermudes	00000236253760	CARLOS ROBERTO CORREA DAS NEVES	Aguardando Visita (Consultor Ativo / S/ Compra)	Recuperação (Fatura Potenci